Universidade de Aveiro
Departamento de Electrónica, Telecomunicações e Informática

Security in Software Engineering

Threat Modelling, SAST & DAST Report

Group 102

Tomás Brás Nº 112665
Afonso Ferreira Nº 113480

June 11, 2026

Contents

Chapter 1

Threat Modelling – Context and Objectives

This threat model covers the full-stack clinic management application composed of a React/Vite frontend, a Spring Boot backend, and a PostgreSQL database, orchestrated via Docker Compose. The system manages patient and appointment data for a medical clinic.

Security objectives:

- Protect patient health and contact data (PII/PHI) from unauthorised disclosure.
- Prevent unauthorised creation, modification, or deletion of patient and appointment records.
- Maintain availability of the system for clinic staff.
- Ensure data integrity of medical records.

Out of scope: Network-level infrastructure, physical security, and end-user workstations.

1.1 System Description

Table 1.1: System components and exposure levels

Component	Technology	Exposure
Frontend	React 19 + Vite 7, Node dev server	Public (browser)
Backend	Java 21 + Spring Boot 4.0.2, port 8080	Host-exposed, proxied via Vite
Database	PostgreSQL 16	Internal only
Orchestration	Docker Compose	Host only

Key observations with security relevance:

- No authentication or authorisation layer exists anywhere in the baseline stack.

- Backend binds JPA entities directly from HTTP request bodies (no DTOs, no `@Valid`).
- Frontend runs as a dev server (`npm run dev`) inside Docker – not a production build.
- Auto-seeder populates 120 patients and 520 appointments on empty database startup.
- `patientId` is passed as a query parameter when creating appointments.
- `PUT /api/appointments/{id}` allows re-assigning a patient via body `{"patient":{"id":X}}`.
- `PatientRepository.findByEmail()` exists but is unused – email uniqueness is not enforced.
- SPECIALTIES and STATUSES are duplicated between frontend and backend with no server-side enum validation.
- No server-side pagination, rate limiting, or audit logging is implemented.

1.2 Assets

- **Patient PII** – name, date of birth, phone number, and email.
- **Appointment records** – date and time, specialty, status, and patient linkage.
- **Database credentials** – used by the backend to access PostgreSQL.
- **System availability** – necessary for normal clinic operation.
- **Audit trail** – currently absent, representing an additional risk.

1.3 Trust Boundaries

TB-1: HTTP/Browser Boundary No TLS or authentication is used in the current setup.

TB-2: Vite Proxy Boundary `/api` requests are forwarded without authentication or access checks.

TB-3: JPA/JDBC Boundary The backend connects to PostgreSQL with a single read/write account and no row-level security.

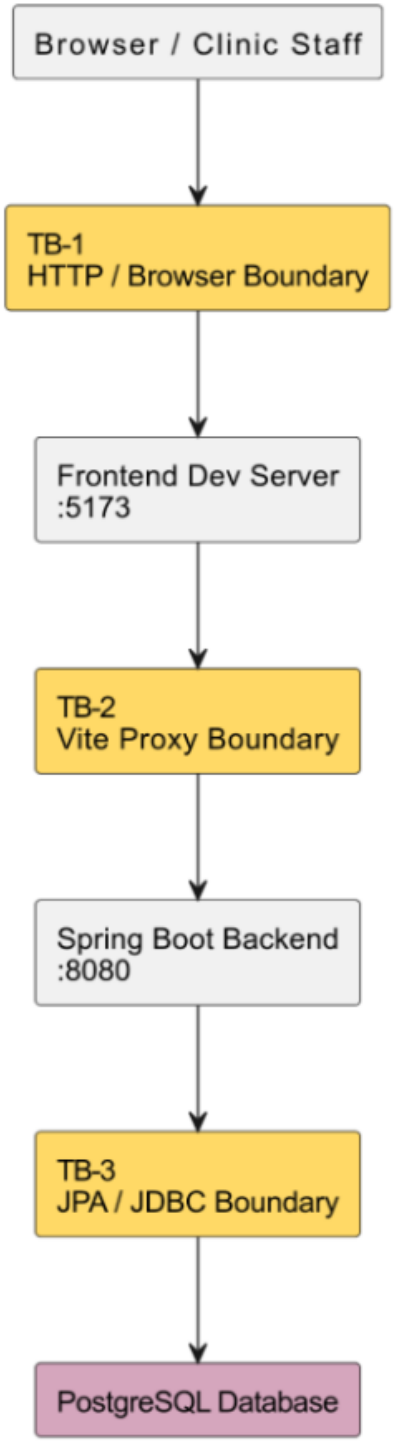


Figure 1.1: System trust boundary stack (TB-1 to TB-3)

1.4 Data Flow Diagrams

1.4.1 DFD Notation

The diagrams use standard DFD-inspired notation:

Rectangle External entity (the Patient).

Circle Process – Manage Patient Data, Manage Appointments, Search and View Information.

Cylinder Data store – D1 Patient Records, D2 Appointment Records.

Arrow Data flow between entities, processes, and stores.

1.4.2 Level 0 – Context Diagram

The Level 0 DFD presents the entire application as a single process. The external entity is the Patient, two trust boundaries are identified (TB-1 between Patient and the system, TB-3 between the system and the database).

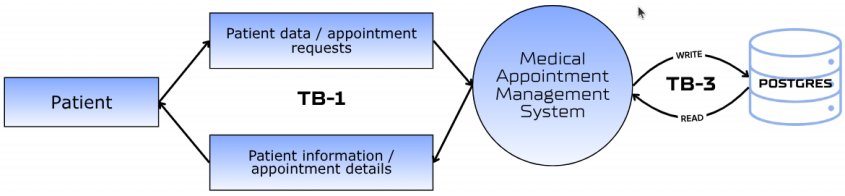


Figure 1.2: DFD Level 0 – Context Diagram

1.4.3 Level 1 – System Decomposition

The Level 1 DFD decomposes the system into three internal sub-processes:

- 1.0 Manage Patient Data
- 2.0 Manage Appointments
- 3.0 Search and View Information

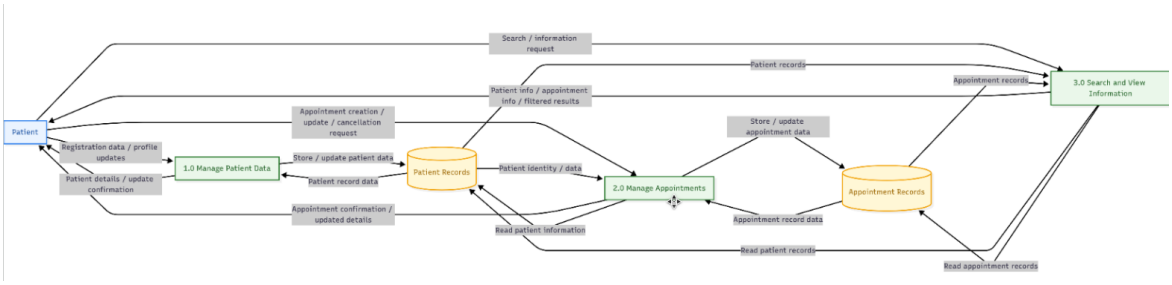


Figure 1.3: DFD Level 1 – System Decomposition

1.4.4 Level 2 – Detailed View of Appointment Creation

The Level 2 view focuses on the Appointment Creation Flow, covering Patient, AppointmentController, AppointmentService, PatientRepository, AppointmentRepository, and Database.

Main steps:

1. The patient sends `POST /api/appointments?patientId=ID`.
2. The controller forwards the request to the service layer.

3. The service validates the patient by checking `patientId`.
4. If the patient exists, the appointment is saved and HTTP 201 is returned.
5. If the patient does not exist, HTTP 404 is returned.

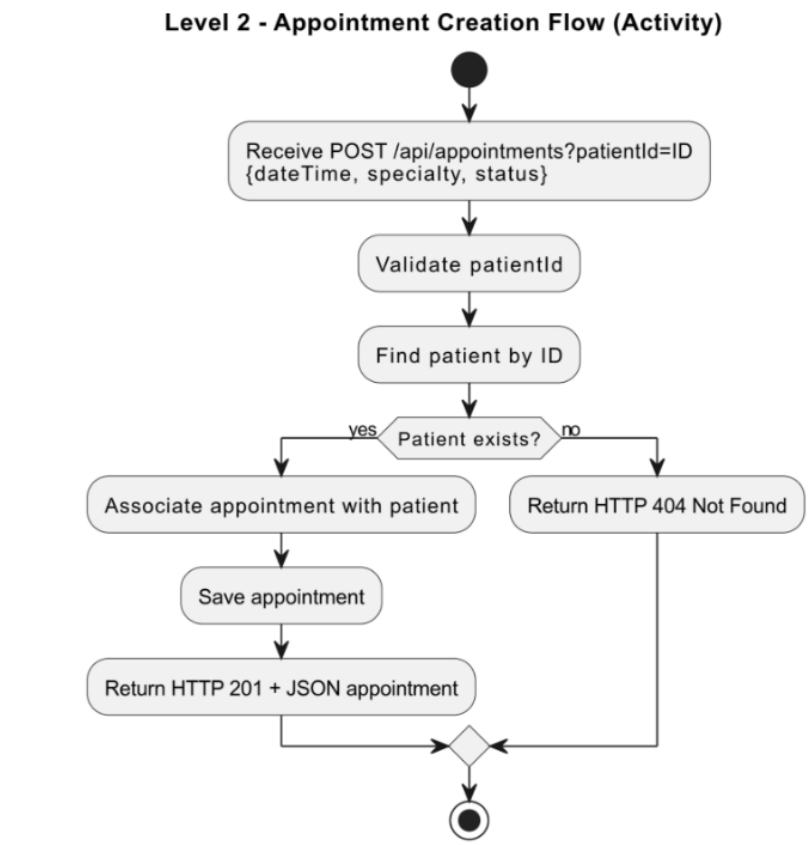


Figure 1.4: DFD Level 2 – Appointment Creation (activity diagram)

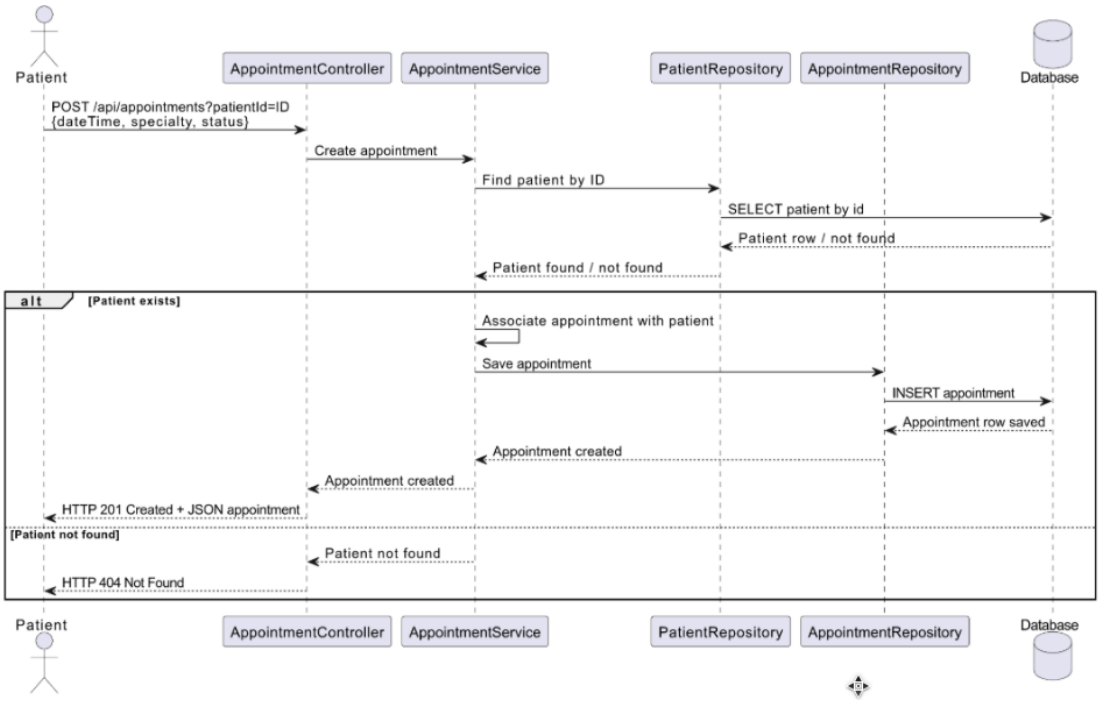


Figure 1.5: DFD Level 2 – Appointment Creation (sequence diagram)

1.5 Approach

This project uses both **LINDDUN** and **STRIDE** as complementary threat modelling methodologies. LINDDUN covers privacy risks (identifiability, linkability, disclosure, inference of health patterns), while STRIDE covers broader security threats (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege).

1.6 LINDDUN Analysis

Table 1.2: LINDDUN privacy threat analysis

Category	Relevance	Description	Detail
Identifiability	High	Patients can be directly identified from stored records (name, email, phone, date of birth).	All fields returned via `GET /api/patients`. Sequential IDs make enumeration trivial.
Linkability	High	Patient identity can be linked to appointment records, enabling inference of health patterns.	Appointment is linked to Patient. Search filters by `patientName`, `date`, `status`, `specialty` – all without authentication.

(continued)

Category	Relevance	Description	Detail
Disclosure of Information	High	Unauthenticated endpoints expose full patient and appointment data.	`GET /api/patients` and `GET /api/appointments` require no credentials.
Detectability	Medium	An attacker can confirm whether a named individual is a patient via API responses.	Sequential IDs and 200/404 differences on `/{id}` endpoints expose record existence.
Non-repudiation	Medium	No authentication or audit logging – data access and modifications cannot be attributed.	No session tracking or audit events. Any CRUD operation is permanently unattributable.
Unawareness	Medium	Patients have no way to know how their data is processed or to exercise their rights.	No consent flow, privacy notice, or rights-handling workflow. GDPR Articles 15–17 cannot be fulfilled.
Non-compliance	High	Structural GDPR violations applicable to health data expose the clinic to regulatory liability.	No lawful basis for processing; no explicit consent; no retention policy; no breach notification.

1.7 STRIDE Threat Analysis

1.7.1 Spoofing

Table 1.3: Spoofing threats

ID	Threat	Component	Severity	Detail
S-01	Any user can impersonate any clinic staff member – **no authentication exists**.	Backend API	**Critical**	No Spring Security configuration or auth middleware is present anywhere in the stack.
S-02	Attacker can forge `patientId` query parameter to create appointments under any patient.	Appointment API	**High**	The controller accepts `patientId` from the request and the service trusts it after `findById()`.

1.7.2 Tampering

Table 1.4: Tampering threats

ID	Threat	Component	Severity	Detail
T-01	Any unauthenticated user can PUT to `/api/appointments/{id}` and change patient linkage, date, status, or specialty.	Appointment API	**Critical**	The appointment update service updates all mutable fields and can silently swap the linked patient.
T-02	Any unauthenticated user can PUT to `/api/patients/{id}` and modify patient PII.	Patient API	**Critical**	The patient update service directly replaces name, date of birth, phone number, and email.
T-03	Direct entity binding accepts nested relationship data from clients.	Backend (no DTOs)	**Medium**	Controllers deserialise directly into JPA entities instead of constrained request DTOs.
T-04	SPECIALTIES and STATUSES not validated server-side – arbitrary strings can be persisted.	Appointment entity	**Medium**	`specialty` and `status` are plain String fields with no enum or bean validation.
T-05	Email field has no uniqueness constraint enforced by the application.	Patient API	**Medium**	`findByEmail()` exists in the repository but is unused during create or update flows.

1.7.3 Repudiation

Table 1.5: Repudiation threats

ID	Threat	Component	Severity	Detail
R-01	No audit logging – any CRUD operation leaves no trace.	Entire backend	**High**	No audit service, interceptor, event sink, or change history is implemented.
R-02	No authentication means no actor identity is ever recorded in logs.	Entire system	**Critical**	Requests are anonymous end-to-end; server logs cannot attribute actions to any real actor.
R-03	Cascade delete on Patient removes all linked appointments with no log or recovery path.	Patient delete API	**High**	Patient uses `CascadeType.ALL` and `orphanRemoval = true` on appointments.

1.7.4 Information Disclosure

Table 1.6: Information Disclosure threats

ID	Threat	Component	Severity	Detail
I-01	Full patient PII returned in all `GET /api/patients` responses – no field-level filtering or RBAC.	Patient API	High	The API returns entity objects directly, including PII and linked appointment collections.
I-02	No pagination – `GET /api/patients` returns all 120+ records in one response.	Patient API	Medium	`PatientService.getAll()` calls `findAll()` with no limit, page, or field projection.
I-03	Unhandled error responses may disclose implementation details.	Backend	Low	No explicit production error-handling policy is configured in the repository.
I-04	Database credentials stored in `.env` file – leaked if repo is made public.	Config	High	`docker-compose.yml` reads `POSTGRES_USER` and `POSTGRES_PASSWORD` from the `.env` file.
I-05	Dev server (`npm run dev`) exposes development assets and implementation details.	Frontend container	Medium	The frontend runs Vite directly instead of a production build through a hardened web server.
I-06	Backend port 8080 may be directly accessible, bypassing the Vite proxy.	Docker networking	Medium	`docker-compose.yml` publishes the backend port to the host, not only the internal Docker network.

1.7.5 Denial of Service

Table 1.7: Denial of Service threats

ID	Threat	Component	Severity	Detail
D-01	No rate limiting – attacker can flood `POST /api/appointments` to exhaust DB connections.	Backend API	High	No throttling, quota, or back-pressure mechanism at any layer.
D-02	No pagination on `GET /api/appointments` – large result sets can exhaust memory.	Appointment search	Medium	The appointment repository returns the full matching set in memory with no page limit.
D-03	`DELETE /api/patients/{id}` cascades to all appointments – one request can delete hundreds of records.	Patient delete	High	A single patient deletion removes the parent record and all dependent appointments.

1.7.6 Elevation of Privilege

Table 1.8: Elevation of Privilege threats

ID	Threat	Component	Severity	Detail
E-01	No roles or permissions – all API operations are equally accessible to any caller.	Entire backend	**Critical**	No RBAC, ownership check, or record-level authorisation anywhere in the backend.
E-02	Client-controlled nested objects can influence backend relationships during updates.	Backend (no DTOs)	**High**	The update flow trusts nested `patient.id` in request bodies to change appointment ownership.
E-03	`PUT /api/appointments/{id}` re-assigns patient – allows transferring medical records between patients.	Appointment service	**High**	The update service resolves the provided patient ID and rebinds the appointment to a different patient if found.

1.8 Abuse Cases

Privacy and security threats were identified using the LINDDUN framework, suited for systems processing personal and health-related data.

LINDDUN categories used: Linking · Identifying · Non-repudiation · Detecting · Disclosure · Unawareness · Non-compliance.

AS-01 – Unauthorised Access to Patient Data

LINDDUN	Disclosure – **Critical**
Abuse Case	As a malicious external user, I want to access another patient's personal data so that I can obtain sensitive information such as name, email, phone number, date of birth, and linked appointment data.
How	The attacker sends direct requests to `/api/patients` and `/api/appointments` and retrieves records without any access control, as no authentication mechanism exists in the system.
Impact	Exposure of personal and appointment-related data, leading to a privacy breach.

AS-02 – Link Patient Identity to Appointment History

LINDDUN	Linking – **Critical**
Abuse Case	As a malicious external user, I want to associate patient identity data with appointment records so that I can infer sensitive patterns such as recurring specialties and possible health-related conditions.
How	The attacker correlates the full patient list with appointment records through linked identifiers. Both endpoints are unauthenticated and return all records without pagination, making full profiling possible in two requests.
Impact	Sensitive health patterns can be inferred from metadata alone, increasing patient profiling risk beyond what any single record reveals.

AS-03 – Directly Identify a Patient from Stored Records

LINDDUN	Identifying – **Critical**
Abuse Case	As a malicious external user, I want to use personal identifiers stored in the system so that I can directly identify a patient and associate their appointment information with a real individual.
How	The attacker accesses records containing direct identifiers (name, email, phone, date of birth). Patient IDs are sequential integers starting at 1, making full enumeration via `/api/patients/{id}` trivial with no rate limiting or authentication.
Impact	A real individual can be directly identified and their full appointment history retrieved.

AS-04 – Infer a Patient's Health Condition from Appointment Metadata

LINDDUN	Detecting – **Critical**
Abuse Case	As a malicious external user, I want to observe the specialty and status fields of a patient's appointments so that I can infer sensitive health information without ever accessing a medical record directly.
How	The attacker filters `GET /api/appointments?patientName=X` to retrieve the full appointment history. Repeated "Psychiatry" visits with status `NO_SHOW` reveal sensitive patterns without any explicit diagnosis data.
Impact	Sensitive health conditions may be inferred from observable metadata, constituting a privacy violation.

AS-05 – Modify or Reassign Patient Appointment Records

LINDDUN	Disclosure – **High**
Abuse Case	As a malicious external user, I want to change the date, status, specialty, or linked patient of an appointment so that I can corrupt legitimate medical records.
How	The attacker sends an unauthenticated PUT to `/api/appointments/{id}`. Including `{"patient":{"id":X}}` in the body causes the service to silently reassign the appointment to a different patient, with no authorisation check or audit log.
Impact	Medical records may be altered or linked to the wrong patient, directly affecting clinical integrity and patient safety.

AS-06 – Destroy Patient Records Through Cascade Deletion

LINDDUN	Non-Compliance – **Critical**
Abuse Case	As a malicious external user, I want to remove patient records so that I can cause irreversible data loss and disrupt normal clinical operation.
How	The attacker sends unauthenticated DELETE requests to `/api/patients/{id}`. Due to cascade deletion (`orphanRemoval=true`), every linked appointment is also permanently destroyed. Sequential IDs allow iterating from 1 to N to wipe the entire database, with no confirmation, audit log, or recovery path.
Impact	All patient PII and linked appointment history is permanently lost with no recovery mechanism. The clinic cannot detect, investigate, or report the incident.

AS-07 – Perform Undetectable Actions Due to Absence of Audit Logging

LINDDUN	Non-repudiation – **Critical**
Abuse Case	As a malicious internal user or external attacker, I want to create, modify, or delete patient and appointment records so that my actions leave no trace and cannot be attributed to me.
How	The system has no authentication, no audit log, and no access log at any layer. Any CRUD operation on any record is permanently undetectable once performed.
Impact	Malicious or accidental changes cannot be investigated or attributed. The clinic cannot comply with GDPR.

AS-08 – Overload the API with Repeated Requests

LINDDUN	Non-compliance – **High**
Abuse Case	As an attacker, I want to flood patient and appointment endpoints with repeated requests so that I can make the system slow or unavailable for legitimate users.
How	The attacker automates repeated API requests. No rate limiting exists, and `GET /api/appointments` with no filters triggers a full table scan with no pagination, exhausting the HikariCP connection pool (∼10 connections default).
Impact	The system becomes slow or unavailable for normal clinical use.

1.9 Attack Trees

Attack trees decompose each high-level attacker goal into concrete, testable attack paths, making the underlying assumptions, preconditions, and impacts explicit. Eight attack trees were produced – one for each abuse story – and they were used to drive the selection and ordering of the pentest scenarios.

1. Unauthorised Access to Patient Data (AS-01)

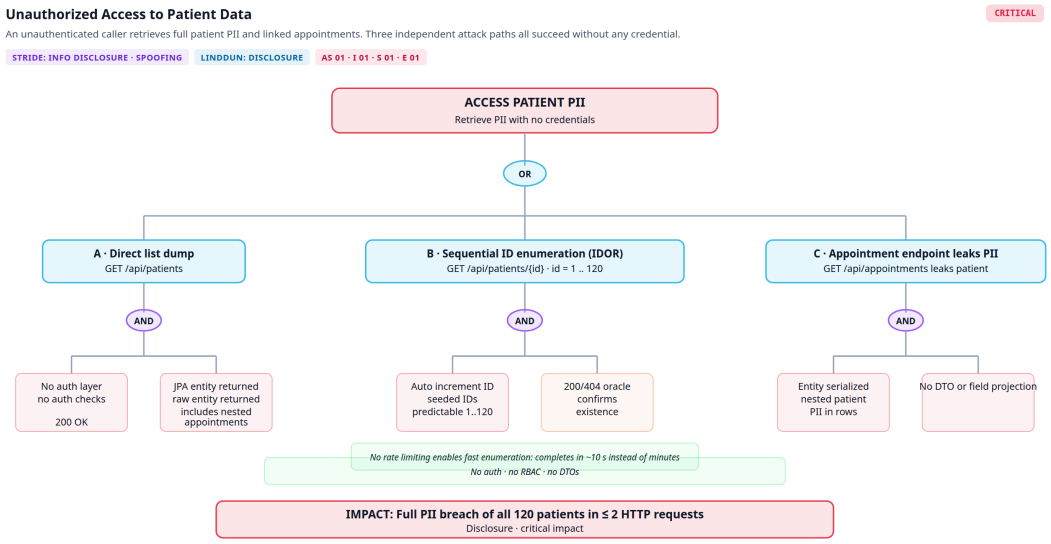


Figure 1.6: Attack tree 1 – Unauthorised Access to Patient Data

Title	Unauthorised Access to Patient Data
Description	An unauthenticated caller retrieves full patient PII and linked appointments. Three independent attack paths all succeed without any credential.
STRIDE	Information Disclosure, Spoofing
LINDDUN	Disclosure
Abuse Stories	AS-01, I-01, S-01, E-01
Severity	**Critical**

2. Link Patient Identity to Health Patterns (AS-02 / AS-04)

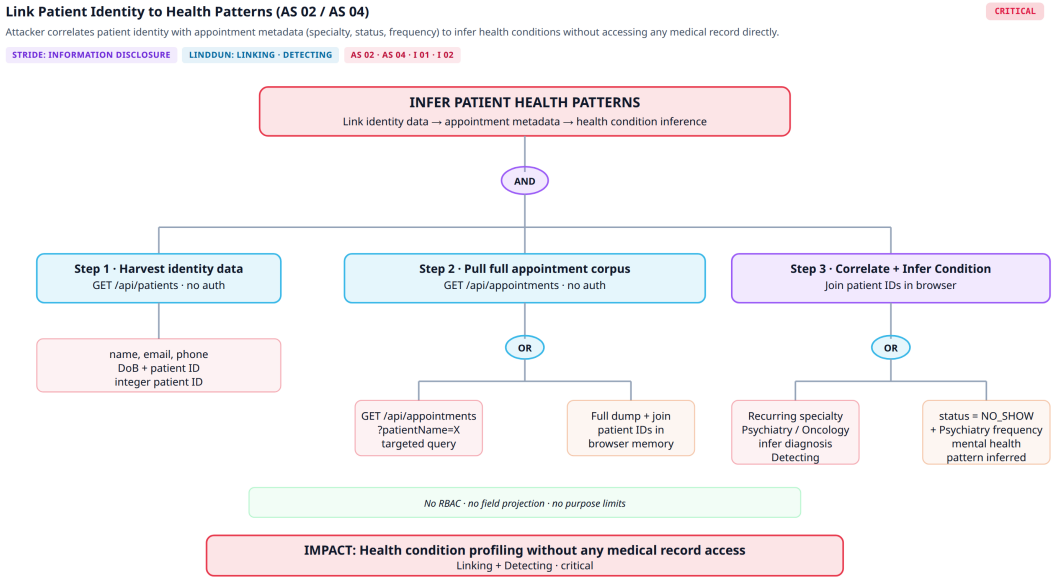


Figure 1.7: Attack tree 2 – Link Patient Identity to Health Patterns

Title	Link Patient Identity to Health Patterns
Description	Attacker correlates patient identity with appointment metadata (specialty, status, frequency) to infer health conditions without accessing any medical record directly.
STRIDE	Information Disclosure
LINDDUN	Linking, Detecting
Abuse Stories	AS-02, AS-04, I-01, I-02
Severity	**Critical**

3. Direct Patient Identification via IDOR (AS-03)

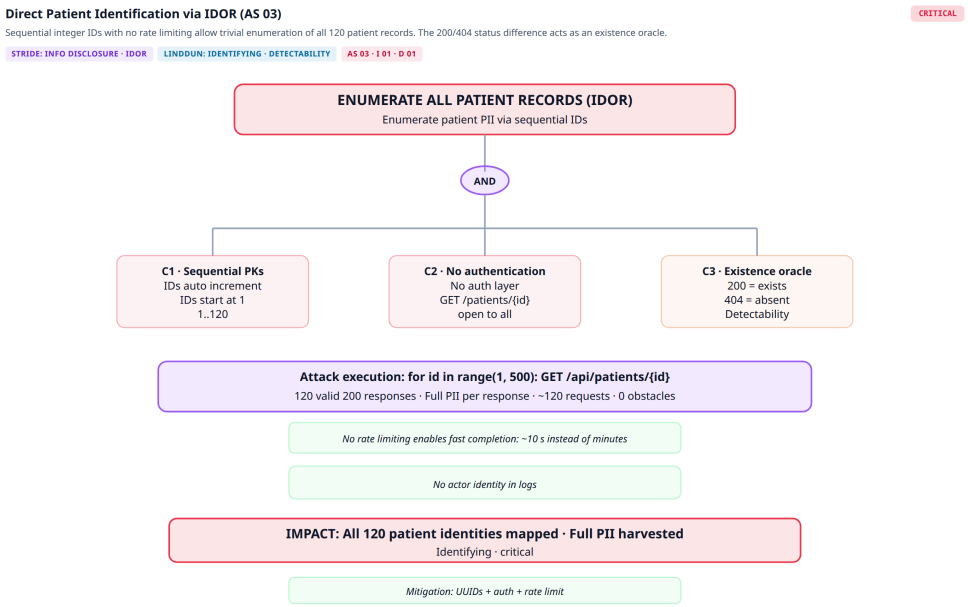


Figure 1.8: Attack tree 3 – Direct Patient Identification via IDOR

Title	Direct Patient Identification via IDOR
Description	Sequential integer IDs with no rate limiting allow trivial enumeration of all 120 patient records. The 200/404 status difference acts as an existence oracle.
STRIDE	Information Disclosure, IDOR
LINDDUN	Identifying, Detectability
Abuse Stories	AS-03, I-01, D-01
Severity	**Critical**

4. Modify or Reassign Appointment Records (AS-05)

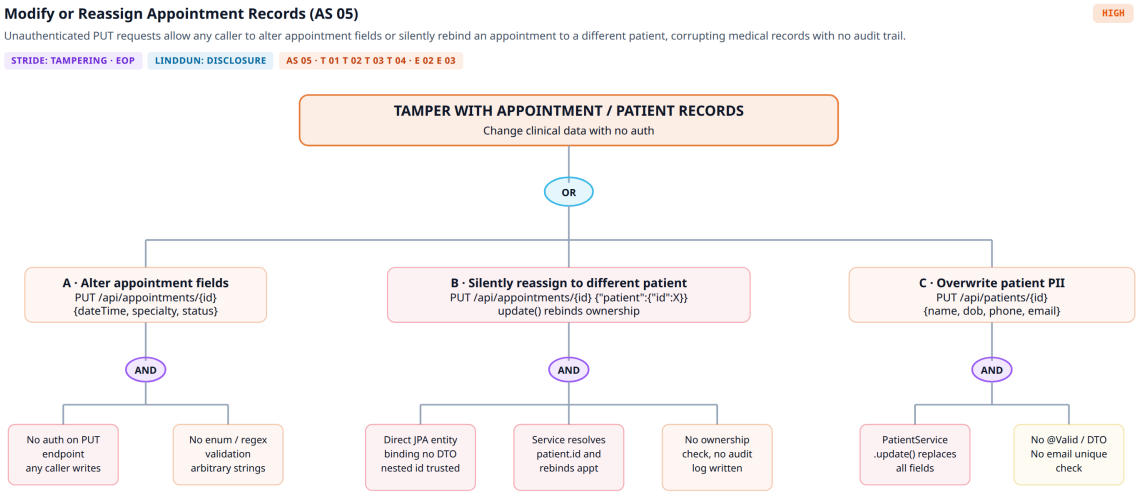


Figure 1.9: Attack tree 4 – Modify or Reassign Appointment Records

Title	Modify or Reassign Appointment Records
Description	Unauthenticated PUT requests allow any caller to alter appointment fields or silently rebind an appointment to a different patient, corrupting medical records with no audit trail.
STRIDE	Tampering, Elevation of Privilege
LINDDUN	Disclosure
Abuse Stories	AS-05, T-01–T-04, E-02, E-03
Severity	**High**

5. Destroy Patient Records via Cascade Deletion (AS-06)

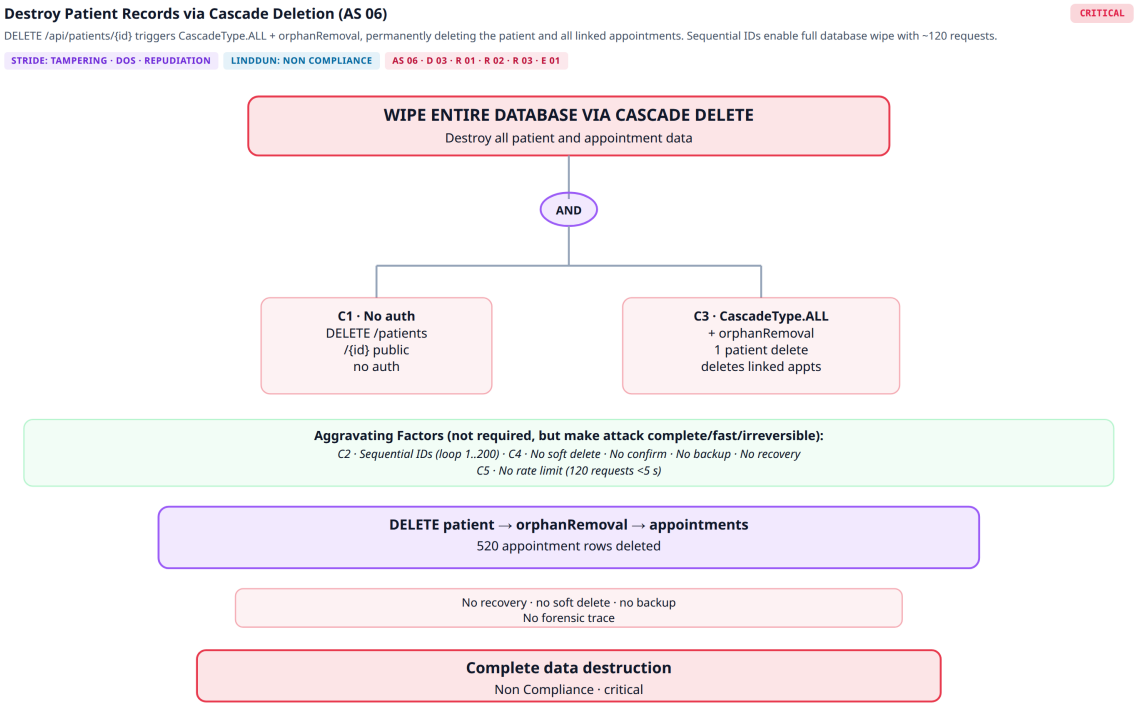


Figure 1.10: Attack tree 5 – Destroy Patient Records via Cascade Deletion

Title	Destroy Patient Records via Cascade Deletion
Description	`DELETE /api/patients/{id}` triggers CascadeType.ALL + orphanRemoval, permanently deleting the patient and all linked appointments. Sequential IDs enable a full database wipe with ∼120 requests.
STRIDE	Tampering, DoS, Repudiation
LINDDUN	Non-Compliance
Abuse Stories	AS-06, D-03, R-01, R-02, R-03, E-01
Severity	**Critical**

6. Perform Undetectable Actions (AS-07)

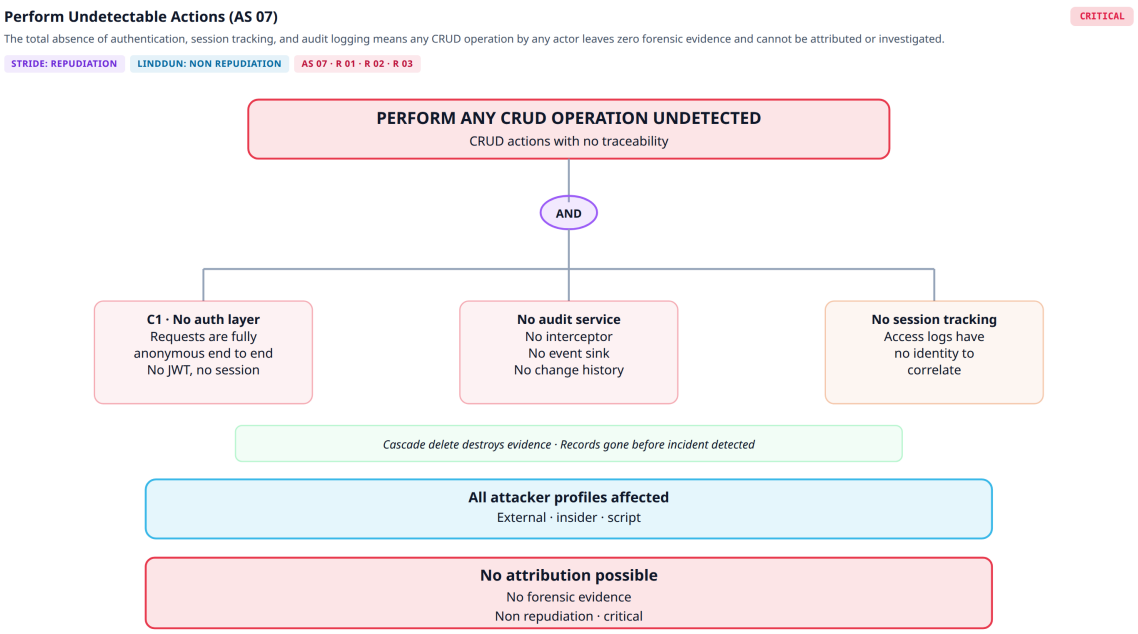


Figure 1.11: Attack tree 6 – Perform Undetectable Actions

Title	Perform Undetectable Actions
Description	The total absence of authentication, session tracking, and audit logging means any CRUD operation leaves zero forensic evidence and cannot be attributed or investigated.
STRIDE	Repudiation
LINDDUN	Non-repudiation
Abuse Stories	AS-07, R-01, R-02, R-03
Severity	Critical

7. Overload API / Denial of Service (AS-08)

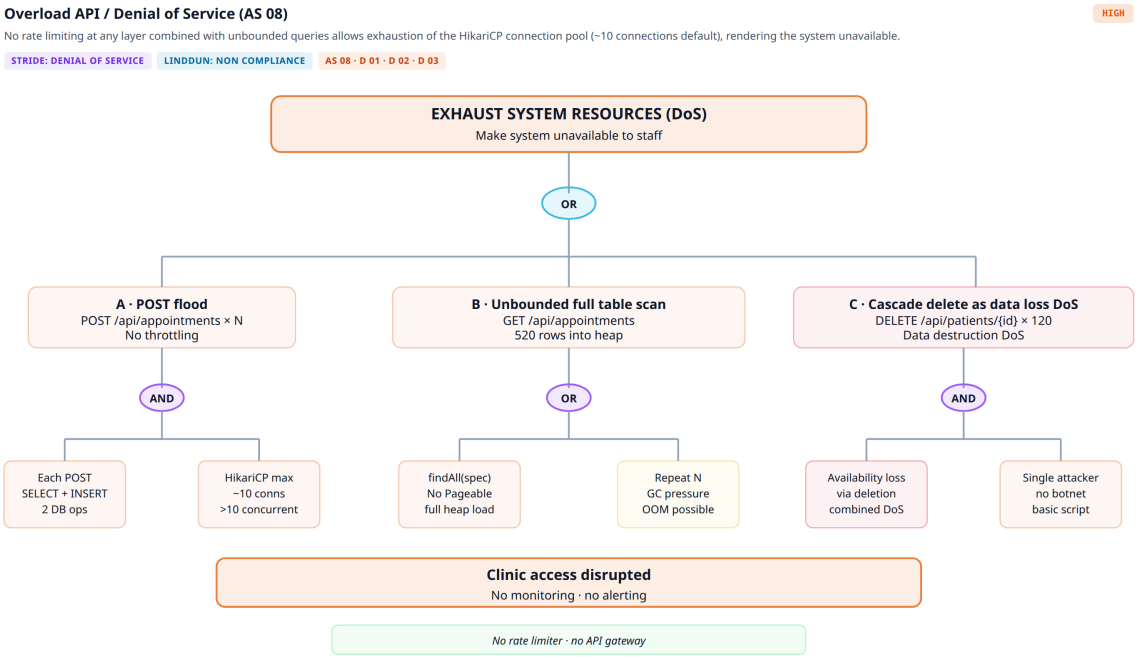


Figure 1.12: Attack tree 7 – Overload API / Denial of Service

Title	Overload API / Denial of Service
Description	No rate limiting at any layer combined with unbounded queries allows exhaustion of the HikariCP connection pool (∼10 connections default), rendering the system unavailable.
STRIDE	Denial of Service
LINDDUN	Non-Compliance
Abuse Stories	AS-08, D-01, D-02, D-03
Severity	High

Chapter 2

SAST – Static Application Security Testing

Static Application Security Testing (SAST) was used in this project to analyse source code, security configuration, and build artefacts without relying on runtime exploitation. Its purpose is to identify insecure coding patterns, exposed secrets, vulnerable dependencies, and insecure project configurations as early as possible in the development lifecycle.

For this project, SAST is organised as a three-level process:

- **pre-commit** – fast local feedback before code is committed;
- **pull request** – gated review before changes are merged;
- **nightly** – broader recurring analysis with deeper coverage.

2.1 Objectives

- Detect insecure code patterns in the Java Spring Boot backend and React/Vite frontend.
- Identify secrets, dependency risks, and misconfigurations before deployment.
- Prevent obvious security issues from reaching the main branch.
- Provide evidence through GitHub Actions runs, uploaded artefacts, and summary reports.
- Support the wider application security assessment together with DAST and manual pentesting.

2.2 Scope

The static analysis scope covers the backend source code in `src/backend`, the frontend source code in `src/frontend`, the repository-level security configuration, the GitHub Actions workflow logic, and Docker-related build artefacts.

2.3 Three-Level SAST Strategy

2.3.1 Pre-Commit Level

Defined through `.pre-commit-config.yaml`. Uses:

- **Gitleaks** – detects secrets and sensitive tokens in staged changes.
- **Semgrep** – fast security-focused scan using `p/security-audit` and custom rules in `.github/config/security/semgrep.yml`.

This level is intentionally lightweight and optimised for quick developer feedback.

2.3.2 Pull Request Level

Runs on pull requests through `sast.yml`. Performs:

- **Gitleaks** – over the Git range introduced by the PR.
- **Semgrep** – enforcing scan for ERROR findings; advisory for WARNING and INFO.
- **ESLint Security** – frontend-oriented security linting.
- **Trivy** – filesystem and backend dependency scans.
- **SonarQube** – with a project-specific quality gate.

SonarQube quality gate rules:

- No new bugs; no new vulnerabilities.
- All new security hotspots must be reviewed.
- No significant maintainability degradation in new code.
- Duplication in new code must stay below 3%.

A **SAST Security Summary** is posted back to the pull request.

2.3.3 Nightly Level

Runs through `sast-nightly.yml`. Performs:

- **CodeQL** – for Java/Kotlin and JavaScript/TypeScript.
- **Semgrep** – broader rule sets: `p/owasp-top-ten`, `p/java`, `p/javascript`, and custom rules.
- **Trivy** – image scanning over the backend and frontend Docker images.

2.4 Why the Three Levels Matter

- **pre-commit** minimises developer mistakes before code is pushed.
- **pull request** provides security gating and reviewer visibility before merge.
- **nightly** provides deeper scheduled assurance and broader long-term monitoring.

2.5 Strengths and Limitations

Strengths: Security checks are distributed across different development lifecycle moments rather than concentrated in a single scan. GitHub Actions artefacts, workflow logs, and generated summaries make results easier to review and present as evidence.

Limitations: Static analysis can produce false positives and some findings may not be exploitable in practice. The pre-commit layer also depends on developers having the tooling correctly installed locally. SAST alone cannot fully assess runtime behaviour or business logic flaws, so it should be seen as one component of a broader security assessment.

2.6 Results

2.6.1 Pre-Commit Results

The pre-commit hook was executed successfully via `pre-commit run -all-files`. Both Gitleaks and Semgrep passed with no blocking findings, confirming no secrets or high-severity patterns in the staged codebase.

```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.30.1
    hooks:
      - id: gitleaks
        args:
          - --config=.github/config/security/gitleaks.toml

  - repo: https://github.com/semgrep/pre-commit
    rev: v1.145.0
    hooks:
      - id: semgrep
        name: semgrep (security - fast)
        args:
          - --config
          - p/security-audit
          - --config
          - .github/config/security/semgrep.yml
          - --severity
          - ERROR
          - --error
          - --disable-version-check
          - --metrics=off
          - --use-git-ignore
```

Figure 2.1: Pre-commit hook configuration (`.pre-commit-config.yaml`) – Gitleaks v8.30.1 and Semgrep v1.145.0

2.6.2 Pull Request Results

Table 2.1: SAST Security Summary – PR level

Tool	Gate Status	Advisory Findings
Gitleaks	Passed	–
Semgrep	Passed	1 medium
ESLint Security	Passed	2 warnings
Trivy	Passed	2 low
SonarQube	Passed	0 bugs, 0 vulnerabilities, 0 hotspots

Non-blocking advisory findings:

- **Semgrep (medium)** – dynamic URL used with `urllib` in `.github/scripts/build_sonarqube_pr_comment.py` (line 24).
- **ESLint (2 warnings)** – Generic Object Injection Sink in `src/frontend/src/components/AppointmentsPanel.jsx`:145 and `src/frontend/src/constants/clinicOptions.js`:32.
- **Trivy (2 low)** – Missing HEALTHCHECK instruction in both Dockerfiles (DS-0026).

SonarQube passed with no bugs, vulnerabilities, or security hotspots detected in the new code.

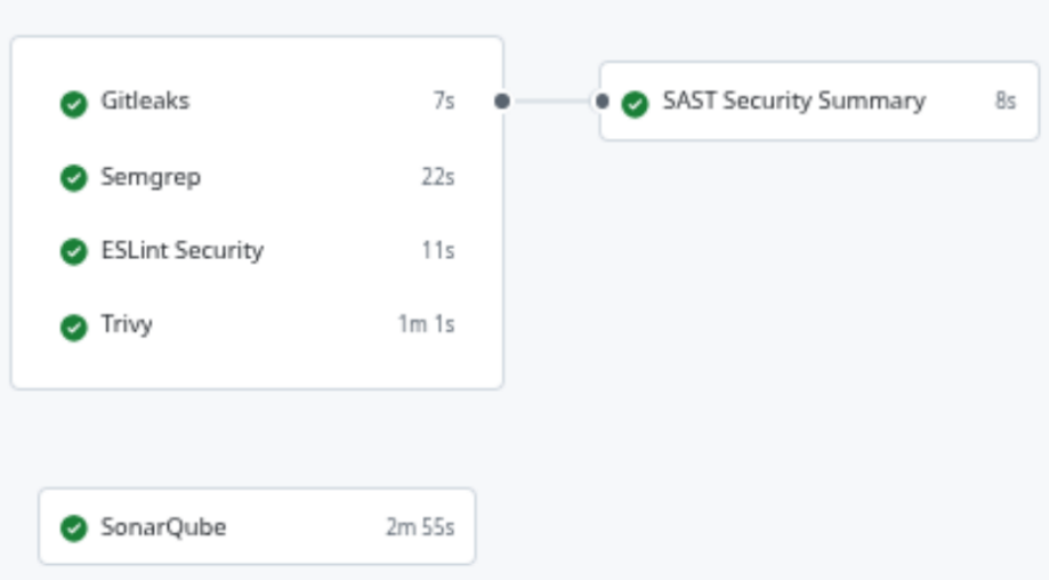
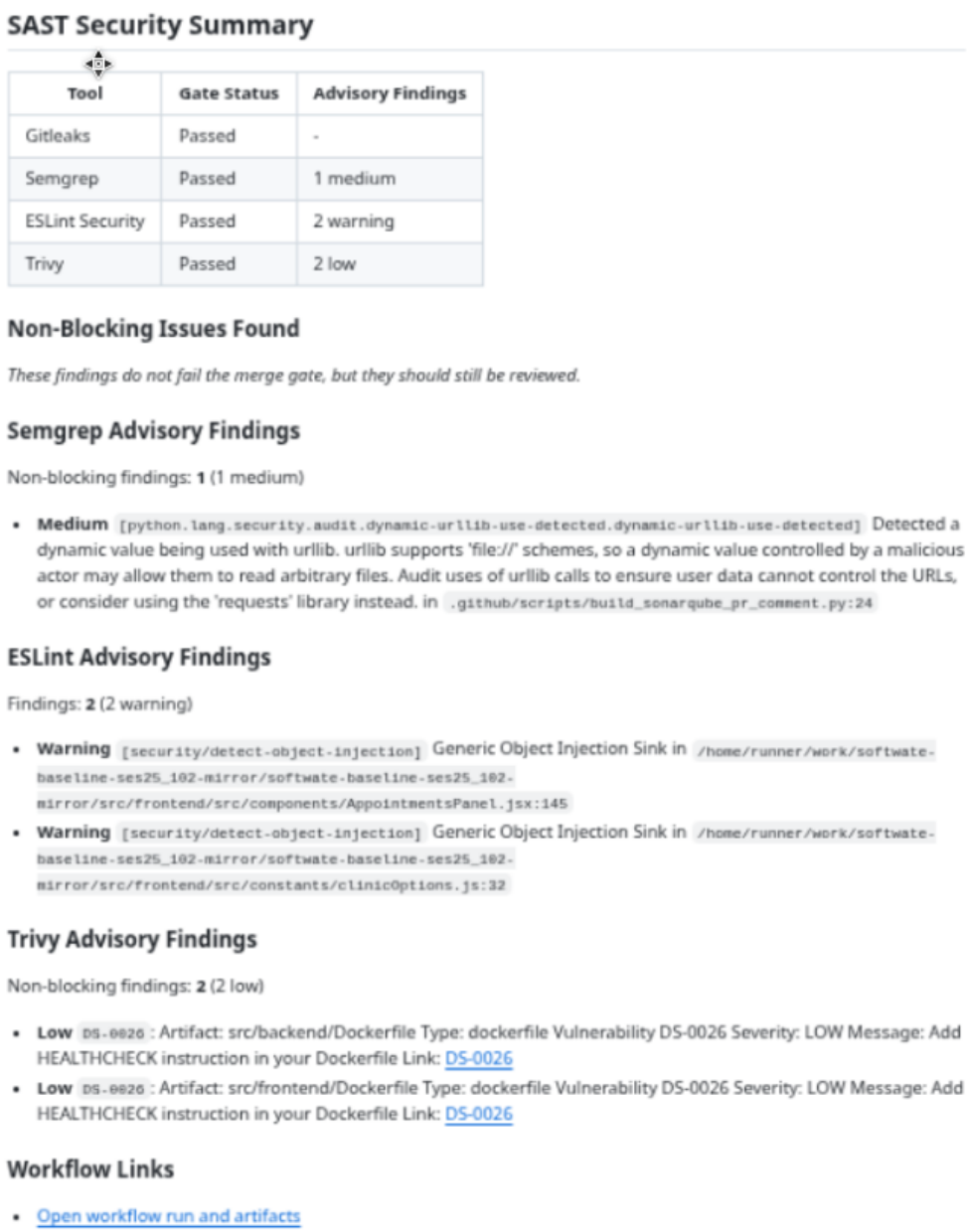


Figure 2.2: GitHub Actions PR workflow – all jobs passed (Gitleaks 7s, Semgrep 22s, ESLint 11s, Trivy 1m1s, SonarQube 2m55s)



SAST Security Summary

Tool	Gate Status	Advisory Findings
Gitleaks	Passed	-
Semgrep	Passed	1 medium
ESLint Security	Passed	2 warning
Trivy	Passed	2 low

Non-Blocking Issues Found

These findings do not fail the merge gate, but they should still be reviewed.

Semgrep Advisory Findings

Non-blocking findings: **1** (1 medium)

- **Medium** `[python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected]` Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a dynamic value controlled by a malicious actor may allow them to read arbitrary files. Audit uses of urllib calls to ensure user data cannot control the URLs, or consider using the 'requests' library instead. in `.github/scripts/build_sonarqube_pr_comment.py:24`

ESLint Advisory Findings

Findings: **2** (2 warning)

- **Warning** `[security/detect-object-injection]` Generic Object Injection Sink in `/home/runner/work/softwate-baseline-ses25_102-mirror/softwate-baseline-ses25_102-mirror/src/frontend/src/components/AppointmentsPanel.jsx:145`
- **Warning** `[security/detect-object-injection]` Generic Object Injection Sink in `/home/runner/work/softwate-baseline-ses25_102-mirror/softwate-baseline-ses25_102-mirror/src/frontend/src/constants/clinicOptions.js:32`

Trivy Advisory Findings

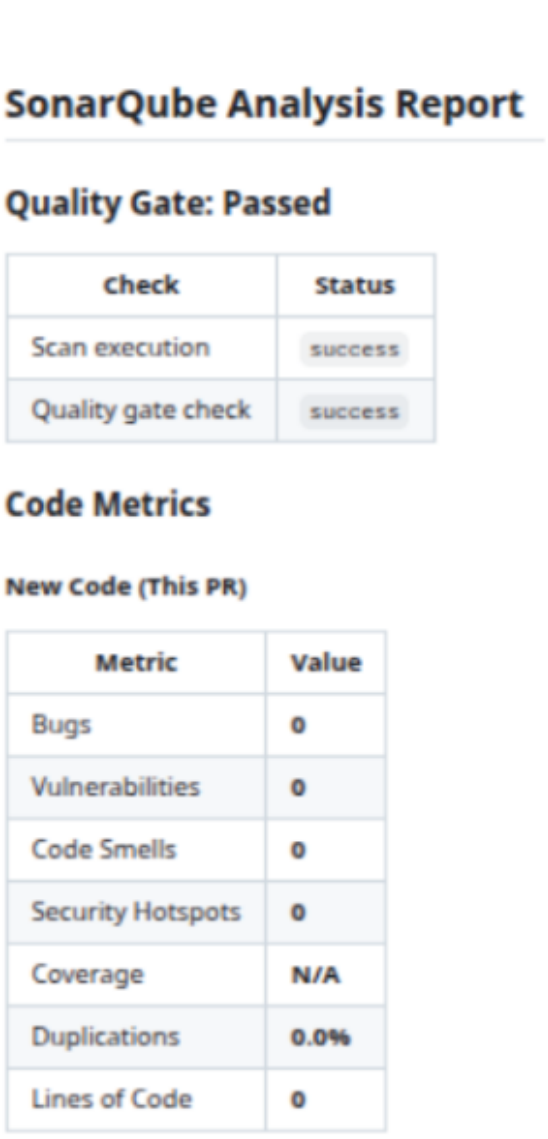
Non-blocking findings: **2** (2 low)

- **Low** `DS-0026` : Artifact: src/backend/Dockerfile Type: dockerfile Vulnerability DS-0026 Severity: LOW Message: Add HEALTHCHECK instruction in your Dockerfile Link: DS-0026
- **Low** `DS-0026` : Artifact: src/frontend/Dockerfile Type: dockerfile Vulnerability DS-0026 Severity: LOW Message: Add HEALTHCHECK instruction in your Dockerfile Link: DS-0026

Workflow Links

- Open workflow run and artifacts

Figure 2.3: SAST Security Summary PR comment – advisory findings per tool



SonarQube Analysis Report

Quality Gate: Passed

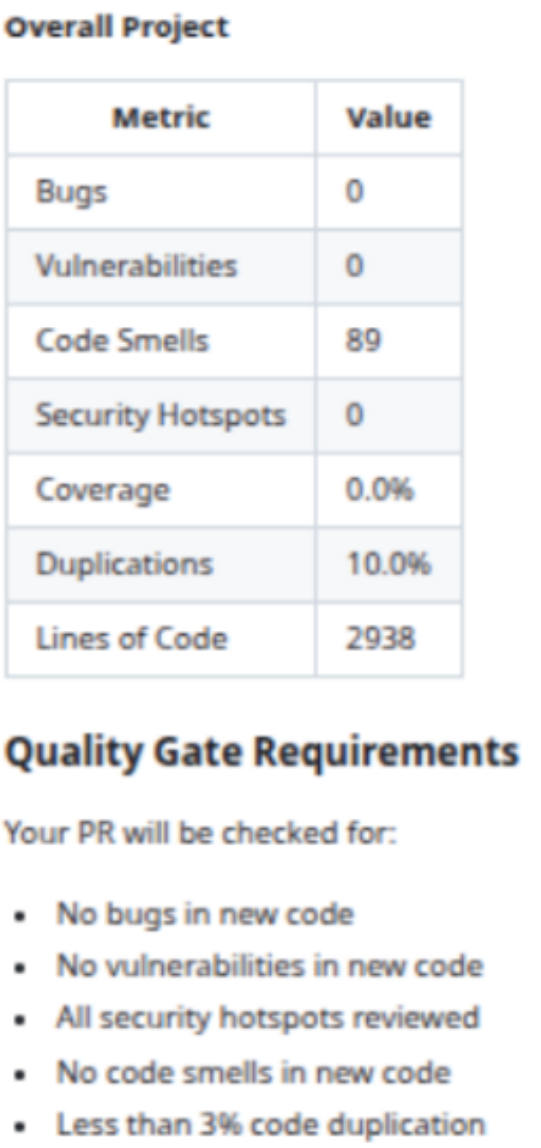
Check	Status
Scan execution	success
Quality gate check	success

Code Metrics

New Code (This PR)

Metric	Value
Bugs	0
Vulnerabilities	0
Code Smells	0
Security Hotspots	0
Coverage	N/A
Duplications	0.0%
Lines of Code	0

Figure 2.4: SonarQube Analysis Report – Quality Gate: Passed (new code metrics)



Overall Project

Metric	Value
Bugs	0
Vulnerabilities	0
Code Smells	89
Security Hotspots	0
Coverage	0.0%
Duplications	10.0%
Lines of Code	2938

Quality Gate Requirements

Your PR will be checked for:

- No bugs in new code
- No vulnerabilities in new code
- All security hotspots reviewed
- No code smells in new code
- Less than 3% code duplication

Figure 2.5: SonarQube – Overall Project metrics (2938 LoC, 89 code smells)

Figure 2.6: PR artifacts – reports uploaded by the SAST pipeline per tool

2.6.3 Nightly Results

CodeQL: No problems found. Scanned: GitHub Actions (3/3), Java (10/11), JavaScript (9/9).

Semgrep: Status Passed – 0 findings across all severities.

Trivy Image Scan:

Table 2.2: Trivy image scan – nightly

Image	Status	Notable CVEs
Backend	Failed	CVE-2025-24734 (HIGH) – `tomcat-embed-core` 11.0.15, fixed in 11.0.18
Frontend	Failed	libcrypto3 CVE-2025-15467 (CRITICAL), libssl3, libc6, and multiple npm packages with HIGH CVEs

The nightly level confirms the value of deeper scheduled analysis: even though pre-commit and PR stages passed, Trivy image scanning detected significant vulnerabilities in the container image layers that were not visible at static code level.

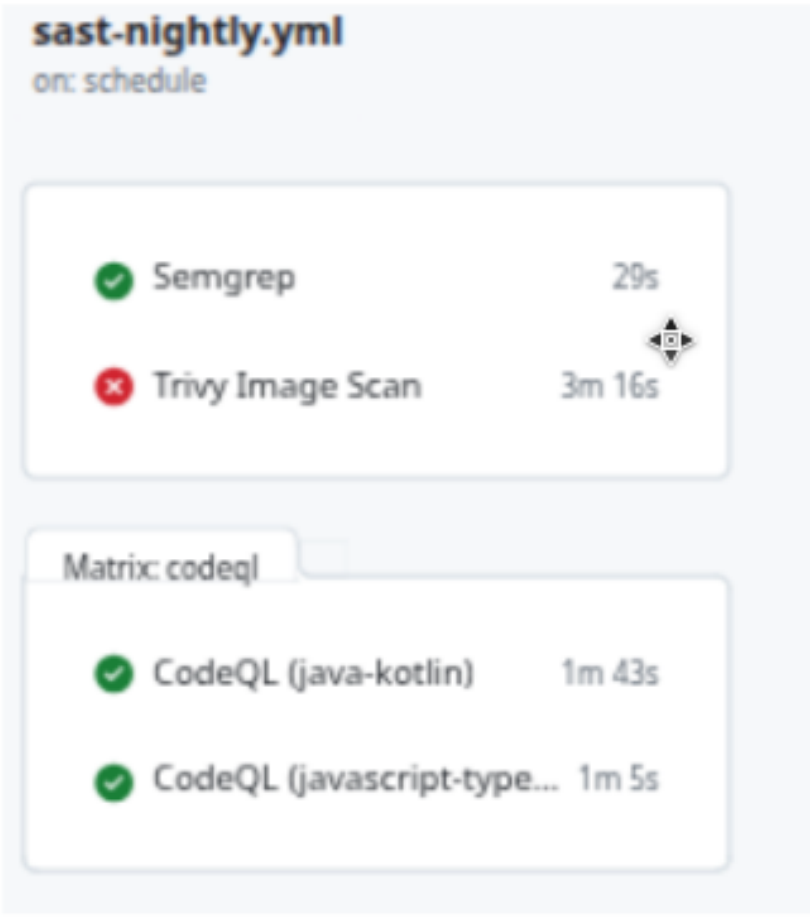


Figure 2.7: Nightly SAST workflow – Semgrep (29s), Trivy Image Scan (3m16s), CodeQL Java-Kotlin (1m43s), CodeQL JavaScript (1m5s)

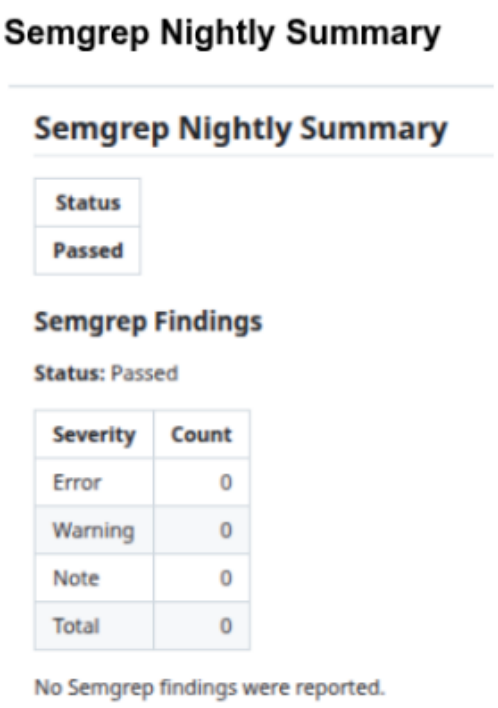


Figure 2.8: Semgrep Nightly Summary – Status: Passed, 0 findings

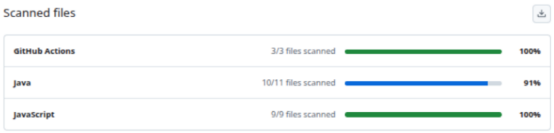


Figure 2.9: CodeQL scanned files – GitHub Actions 100%, Java 91%, JavaScript 100%

Trivy Image Scan Results

Trivy Image Scan Summary

Image	Status
Backend	**failed**
Frontend	**failed**

Backend Image Vulnerabilities

Package	Vulnerability	Severity	Installed	Fixed
org.apache.tomcat.embed:tomcat-embed-core	CVE-2026-24734	HIGH	11.0.15	11.0.18, 10.1.52, 9.0.115

Frontend Image Vulnerabilities

Package	Vulnerability	Severity	Installed	Fixed
libcrypto3	CVE-2025-15467	CRITICAL	3.5.4-r0	3.5.5-r0
libcrypto3	CVE-2025-69419	HIGH	3.5.4-r0	3.5.5-r0
libcrypto3	CVE-2025-69421	HIGH	3.5.4-r0	3.5.5-r0
libssl3	CVE-2025-15467	CRITICAL	3.5.4-r0	3.5.5-r0
libssl3	CVE-2025-69419	HIGH	3.5.4-r0	3.5.5-r0
libssl3	CVE-2025-69421	HIGH	3.5.4-r0	3.5.5-r0
zlib	CVE-2026-22184	HIGH	1.3.1-r2	1.3.2-r0
cross-spawn	CVE-2024-21538	HIGH	7.0.3	7.0.5, 6.0.6
flatted	CVE-2026-32141	HIGH	3.3.3	3.4.0
flatted	CVE-2026-33228	HIGH	3.3.3	3.4.2
glob	CVE-2025-64756	HIGH	10.4.2	11.1.0, 10.5.0
minimatch	CVE-2026-26996	HIGH	3.1.2	10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3
minimatch	CVE-2026-27903	HIGH	3.1.2	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3
minimatch	CVE-2026-27904	HIGH	3.1.2	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4
minimatch	CVE-2026-26996	HIGH	9.0.5	10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3
minimatch	CVE-2026-27903	HIGH	9.0.5	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3
minimatch	CVE-2026-27904	HIGH	9.0.5	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4
picomatch	CVE-2026-33671	HIGH	4.0.3	4.0.4, 3.0.2, 2.3.2
rollup	CVE-2026-27606	HIGH	4.57.1	2.80.0, 3.30.0, 4.59.0
tar	CVE-2026-23745	HIGH	6.2.1	7.5.3

Figure 2.10: Trivy Image Scan Results – Backend (HIGH: tomcat-embed-core CVE-2025-24734) and Frontend (CRITICAL: libcrypto3 CVE-2025-15467 and multiple HIGH CVEs)

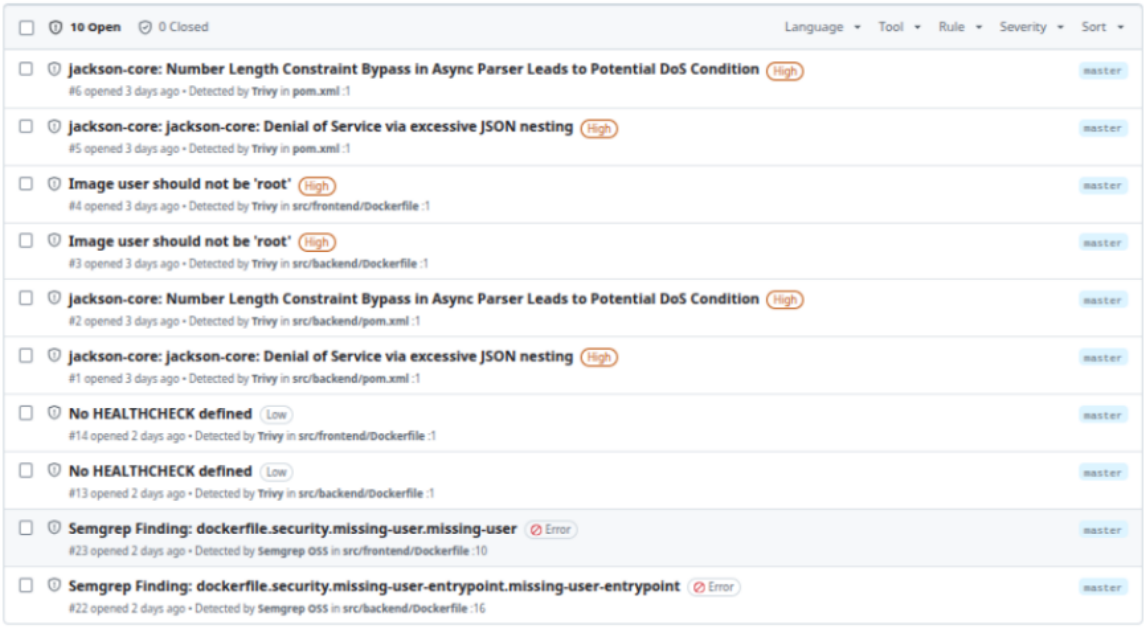


Figure 2.11: GitHub Security tab – open alerts detected by Trivy (HIGH, pom.xml and Dockerfiles) and Semgrep (errors)

Chapter 3

Dynamic Application Security Testing (DAST)

3.1 Overview

Dynamic Application Security Testing (DAST) assesses the running application from the outside, through its exposed web interface and HTTP endpoints, without relying on access to source code. Its purpose is to identify runtime security issues such as missing security headers, weak transport settings, unsafe default behaviour, exposed administrative functionality, and other vulnerabilities that become visible only when the application is deployed and reachable.

For this project, DAST is organised as a two-level process: a pull request level for merge-time validation, and a nightly level for recurring coverage against a stable target branch.

3.2 Objectives

- Detect runtime security weaknesses in the deployed application.
- Validate the behaviour of the frontend and backend when exposed through HTTP.
- Provide merge-time and recurring feedback through GitHub Actions.
- Support the wider application security assessment together with SAST and manual pentesting.

3.3 Scope

The dynamic analysis scope covers the running ClinicWave application deployed in an ephemeral Docker-based environment. This includes the React/Vite frontend, the Spring Boot backend, the PostgreSQL database, the HTTP routes exposed by the frontend, and the backend endpoints reachable through the application during scanning.

3.4 Two-Level DAST Strategy

3.4.1 Pull Request Level

Runs through `dast-pr.yml`. The workflow builds the frontend and backend from the PR branch, starts a temporary PostgreSQL container, brings the full application up inside a dedicated Docker network, waits for both services to become reachable, and then runs an OWASP ZAP Baseline scan against the frontend entry point.

The PR gate is configured to fail when the ZAP report contains High severity findings. The workflow also generates both HTML and JSON ZAP reports, uploads them as workflow artefacts, evaluates the JSON output, and posts a DAST summary comment back to the pull request.

3.4.2 Nightly Level

Runs through `dast-nightly.yml`. This workflow checks out the software repository, builds and launches the full stack inside an isolated Docker environment, and scans with OWASP ZAP Baseline against the running frontend. It produces HTML and JSON ZAP reports, uploads them as artefacts, and writes a summarised view of alert counts and alert types into the workflow summary.

3.4.3 Why the Two Levels Matter

The pull request level provides merge-time validation and direct reviewer visibility. The nightly level provides recurring security monitoring against a stable target branch. This separation is useful because runtime scanning is slower and more environment-dependent than static analysis, so it benefits from having one layer optimised for gating and another for ongoing observation.

3.5 Runtime Environment and Tooling

Both workflows use **OWASP ZAP Baseline**. The application is launched inside Docker with a temporary PostgreSQL container, a backend container exposing the Spring Boot API, and a frontend container exposing the React/Vite application. The scanner targets the running frontend entry point.

3.6 Strengths and Limitations

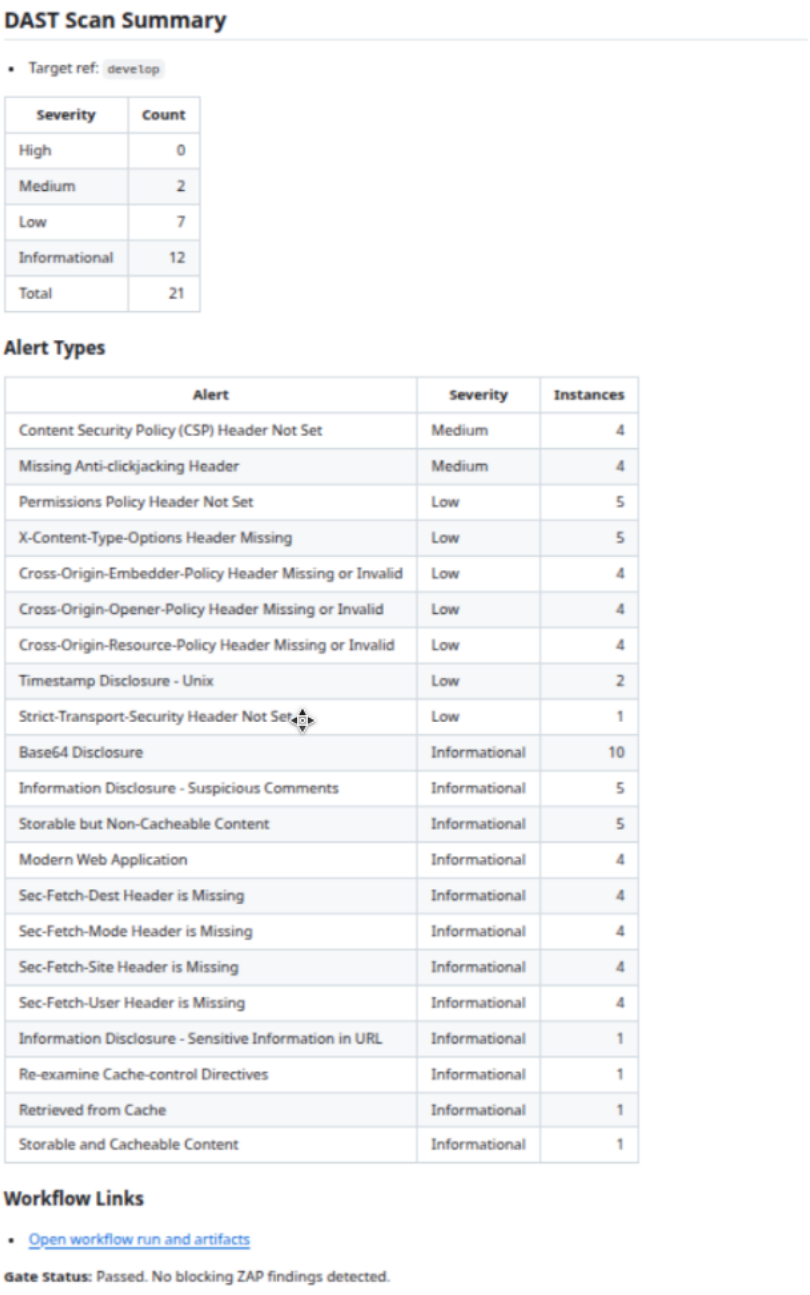
Strengths: DAST validates the application in a running state, identifying issues only visible after deployment (missing headers, weak runtime defaults, exposed routes). GitHub Actions artefacts and PR comments make results easy to review and document.

Limitations: DAST depends on the application starting correctly in the test environment. A ZAP Baseline scan is more effective at identifying broad web security issues than deep business-logic flaws. Results quality depends on how much of the application surface the scanner can reach.

3.7 Results

3.7.1 Pull Request Results

The DAST workflow completed successfully. The gate passed because no **High** severity ZAP findings were detected.



DAST Scan Summary

- Target ref: `develop`

Severity	Count
High	0
Medium	2
Low	7
Informational	12
Total	21

Alert Types

Alert	Severity	Instances
Content Security Policy (CSP) Header Not Set	Medium	4
Missing Anti-clickjacking Header	Medium	4
Permissions Policy Header Not Set	Low	5
X-Content-Type-Options Header Missing	Low	5
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	4
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	4
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	4
Timestamp Disclosure - Unix	Low	2
Strict-Transport-Security Header Not Set	Low	1
Base64 Disclosure	Informational	10
Information Disclosure - Suspicious Comments	Informational	5
Storable but Non-Cacheable Content	Informational	5
Modern Web Application	Informational	4
Sec-Fetch-Dest Header is Missing	Informational	4
Sec-Fetch-Mode Header is Missing	Informational	4
Sec-Fetch-Site Header is Missing	Informational	4
Sec-Fetch-User Header is Missing	Informational	4
Information Disclosure - Sensitive Information in URL	Informational	1
Re-examine Cache-control Directives	Informational	1
Retrieved from Cache	Informational	1
Storable and Cacheable Content	Informational	1

Workflow Links

- Open workflow run and artifacts

Gate Status: Passed. No blocking ZAP findings detected.

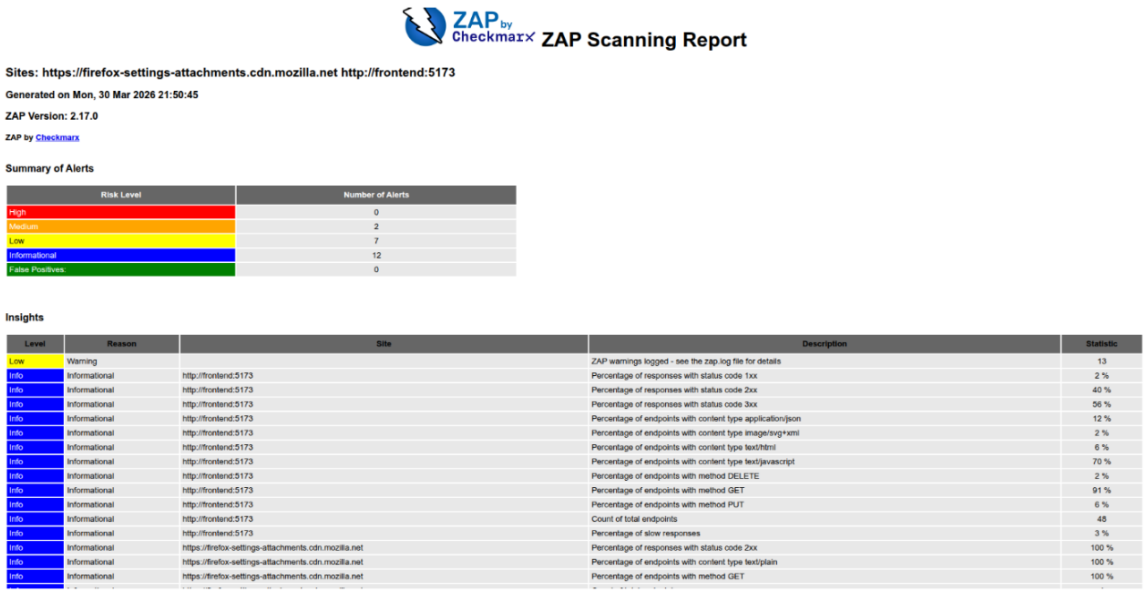
Figure 3.1: DAST PR scan summary comment – no High findings; gate status: Passed

Table 3.1: DAST Scan Summary – PR level

Severity	**Count**
High	0
Medium	2
Low	7
Informational	12
Total	21

Table 3.2: DAST alert types – PR level

Alert	**Severity**	**Instances**
Content Security Policy (CSP) Header Not Set	Medium	8
Missing Anti-clickjacking Header	Medium	4
Permissions Policy Header Not Set	Low	5
X-Content-Type-Options Header Missing	Low	5
Cross-Origin-Embedder-Policy Header Missing	Low	4
Cross-Origin-Opener-Policy Header Missing	Low	4
Cross-Origin-Resource-Policy Header Missing	Low	4
Timestamp Disclosure – Unix	Low	2
Strict-Transport-Security Header Not Set	Low	1
Base64 Disclosure	Informational	10
Information Disclosure – Suspicious Comments	Informational	5
Storable but Non-Cacheable Content	Informational	5
Modern Web Application	Informational	8



ZAP by Checkmarx ZAP Scanning Report

Sites: https://firefox-settings-attachments.cdn.mozilla.net http://frontend:5173

Generated on Mon, 30 Mar 2026 21:50:45

ZAP Version: 2.17.0

ZAP by Checkmarx

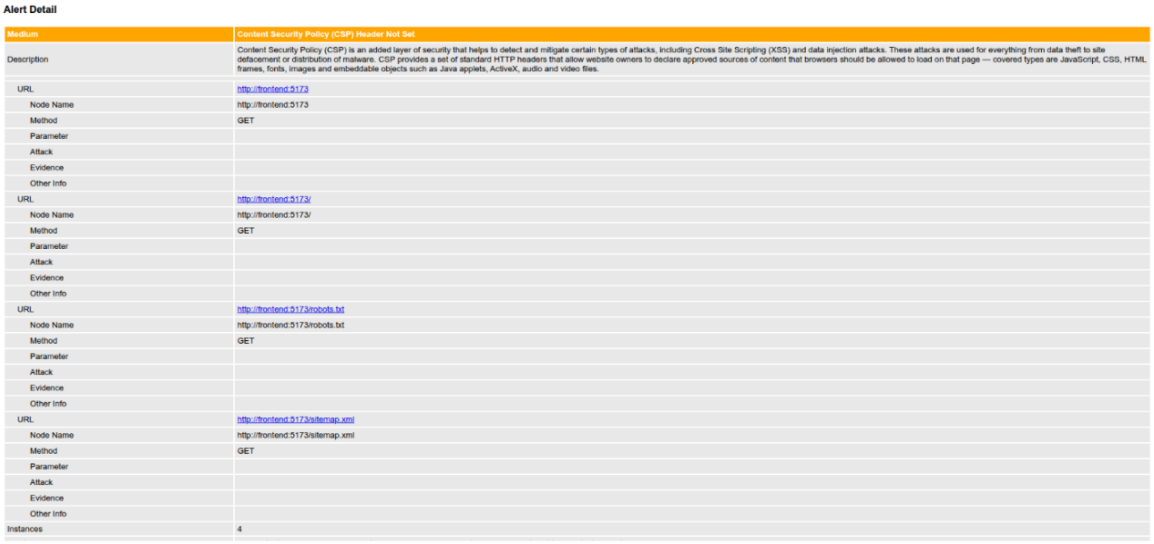
Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	7
Informational	12
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	13
Info	Informational	http://frontend:5173	Percentage of responses with status code 1xx	2 %
Info	Informational	http://frontend:5173	Percentage of responses with status code 2xx	40 %
Info	Informational	http://frontend:5173	Percentage of responses with status code 3xx	56 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type application/json	12 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type image/svg+xml	2 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type text/html	6 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type text/javascript	70 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method DELETE	2 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method GET	91 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method PUT	6 %
Info	Informational	http://frontend:5173	Count of total endpoints	48
Info	Informational	http://frontend:5173	Percentage of slow responses	3 %
Info	Informational	https://firefox-settings-attachments.cdn.mozilla.net	Percentage of responses with status code 2xx	100 %
Info	Informational	https://firefox-settings-attachments.cdn.mozilla.net	Percentage of endpoints with content type text/plain	100 %
Info	Informational	https://firefox-settings-attachments.cdn.mozilla.net	Percentage of endpoints with method GET	100 %

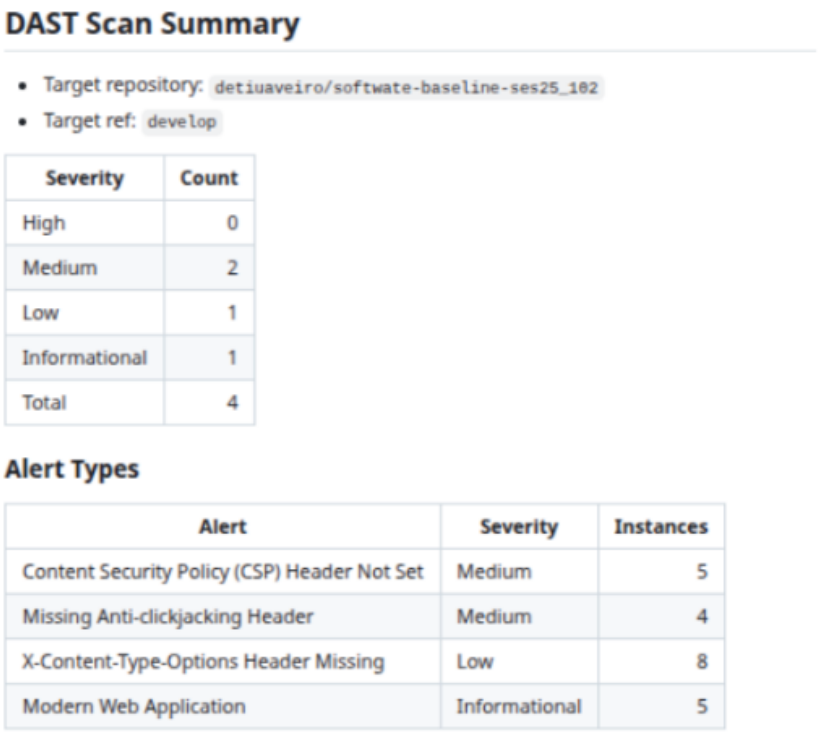
Figure 3.2: ZAP Scanning Report (HTML) – PR run, generated Mon 30 Mar 2026



Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://frontend:5173
Node Name	http://frontend:5173
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://frontend:5173/
Node Name	http://frontend:5173/
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://frontend:5173/robots.txt
Node Name	http://frontend:5173/robots.txt
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://frontend:5173/sitemap.xml
Node Name	http://frontend:5173/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
Instances	4

Figure 3.3: ZAP Alert Detail – Content Security Policy (CSP) Header Not Set (Medium, 4 instances)



DAST Scan Summary

- Target repository: `detiuaveiro/softwate-baseline-ses25_102`
- Target ref: `develop`

Severity	Count
High	0
Medium	2
Low	1
Informational	1
Total	4

Alert Types

Alert	Severity	Instances
Content Security Policy (CSP) Header Not Set	Medium	5
Missing Anti-clickjacking Header	Medium	4
X-Content-Type-Options Header Missing	Low	8
Modern Web Application	Informational	5

Figure 3.4: DAST alert types summary – latest PR scan (4 alert types, High: 0)

3.7.2 Nightly Results

The nightly DAST workflow built and launched the full stack inside Docker and ran OWASP ZAP Baseline against the running frontend.

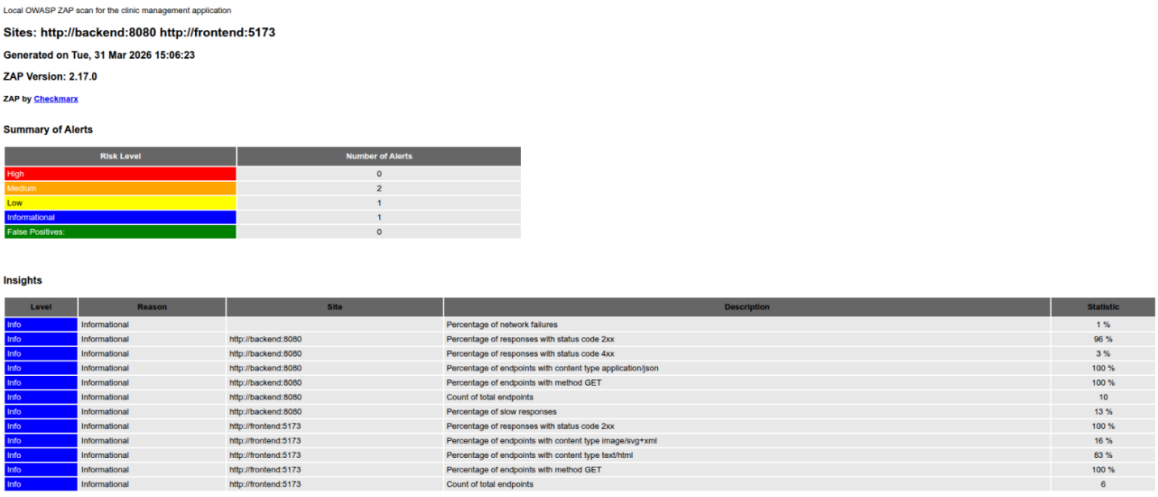
Table 3.3: DAST Scan Summary – Nightly level

Severity	**Count**
High	0
Medium	2
Low	1
Informational	1
Total	4

Table 3.4: DAST alert types – Nightly level

Alert	**Severity**	**Instances**
Content Security Policy (CSP) Header Not Set	Medium	5
Missing Anti-clickjacking Header	Medium	4
X-Content-Type-Options Header Missing	Low	6
Modern Web Application	Informational	5

The nightly results confirm the same top findings as the PR scan. No High severity findings were detected in either run. The nightly level provides a recurring view of the runtime security posture of the project even when no pull request is being reviewed.



Local OWASP ZAP scan for the clinic management application

Sites: http://backend:8080 http://frontend:5173

Generated on Tue, 31 Mar 2026 15:06:23

ZAP Version: 2.17.0

ZAP by Checkmarx

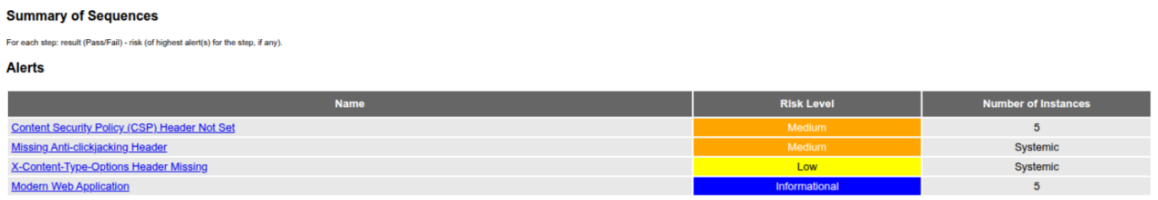
Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	1
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational		Percentage of network failures	1 %
Info	Informational	http://backend:8080	Percentage of responses with status code 2xx	96 %
Info	Informational	http://backend:8080	Percentage of responses with status code 4xx	3 %
Info	Informational	http://backend:8080	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://backend:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://backend:8080	Count of total endpoints	10
Info	Informational	http://backend:8080	Percentage of slow responses	13 %
Info	Informational	http://frontend:5173	Percentage of responses with status code 2xx	100 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type image/svg+xml	16 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type text/html	83 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method GET	100 %
Info	Informational	http://frontend:5173	Count of total endpoints	6

Figure 3.5: Nightly ZAP Scanning Report (HTML) – Local OWASP ZAP scan, generated Tue 31 Mar 2026



Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	5
Missing Anti-clickjacking Header	Medium	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Modern Web Application	Informational	5

Figure 3.6: Nightly ZAP Summary of Sequences – alert breakdown: Medium (CSP, Anti-clickjacking), Low (X-Content-Type), Informational (Modern Web App)

Chapter 4

Part 2 – Security Fixes & Architecture

Building on the Part 1 assessment, Part 2 fixes the vulnerabilities identified and introduces a structured security architecture. The baseline had no authentication, no authorisation, no input validation, and no perimeter controls. This chapter documents the changes made by Tomás Brás (Nº 112665) – API hardening, role-based access control, authentication flow documentation, Zero Trust Architecture framing, and abuse story regression – and references the Keycloak/OIDC integration carried out by Afonso Ferreira.

4.1 Authentication & Identity Management

4.1.1 Overview

The baseline had no authentication layer. The new architecture introduces Keycloak 26.2.5 as the external Identity Provider (IdP), using OpenID Connect (OIDC) with the Authorization Code + PKCE (S256) flow. The backend validates every incoming request against a signed JWT (RS256) fetched from the Keycloak JWKS endpoint.

4.1.2 Token-Based Authentication

- **Access tokens** – short-lived JWTs signed with RS256; validated on every request.
- **Refresh tokens** – long-lived; used by `keycloak-js` to silently renew access tokens before expiry. Keycloak handles rotation and reuse detection server-side.
- **CSRF** – the API is stateless and uses Bearer tokens only (no cookies), so CSRF is structurally inapplicable and is explicitly disabled in `SecurityConfig`.

4.1.3 Authentication Flows

Login Flow

1. User opens the app. `keycloak.init({onLoad: 'login-required', pkceMethod: 'S256'})` is called before any React component renders.
2. If no valid session exists, the browser is redirected to the Keycloak login page.

3. The user authenticates (username + password). Keycloak verifies the credential and issues an authorisation code.
4. PKCE: the frontend sends the stored `code_verifier`; Keycloak verifies the `code_challenge` (SHA-256) and returns an access token + refresh token.
5. Tokens are stored in memory by `keycloak-js`. The frontend proceeds to render.
6. Each API call in `httpConsumer.js` adds `Authorization: Bearer <token>` to the request.
7. The Spring Boot backend receives the token, fetches the JWKS from Keycloak lazily (`NimbusJwtDecoder`), validates the RS256 signature, checks the `iss` claim (`JwtValidators.createDefaultWithIssuer`), and extracts roles via `JwtRoleConverter`.

Logout Flow

1. The user clicks the **Logout** button in the top navigation bar.
2. The frontend calls `keycloak.logout()` with a `redirectUri` set to the current application origin.
3. Keycloak invalidates the session server-side (back-channel logout), clears the SSO cookie, and redirects the browser back to the application origin.
4. The app re-enters `keycloak.init()` and redirects to the Keycloak login page.

Token Refresh Flow

1. Before every API request, `httpConsumer.js` calls `await keycloak.updateToken(30)`.
2. If the access token expires within 30 seconds, `keycloak-js` sends the refresh token to Keycloak's token endpoint and obtains a new access token silently.
3. If the refresh token is also expired, `keycloak.login()` is called, redirecting the user back to the login page with no data loss.

Token Revocation

Keycloak handles revocation server-side. When a user logs out, the session (and all tokens associated with it) is invalidated. If an administrator disables or deletes a user in Keycloak, existing tokens will fail issuer/audience validation on the next backend call (or on the next refresh attempt, whichever comes first).

4.1.4 Roles and Realm Configuration

Three roles are defined in the Keycloak realm (`clinic`):

Table 4.1: Keycloak roles and default permissions

Role	Patients	Appointments
ADMIN	READ, CREATE, UPDATE, DELETE	READ, CREATE, UPDATE, DELETE
DOCTOR	READ, UPDATE	READ, UPDATE
RECEPTIONIST	READ, CREATE	READ, CREATE, UPDATE, DELETE

Users (`admin`, `doctor`, `receptionist`) are assigned to their corresponding realm roles in the Keycloak admin console. The `clinic-frontend` client uses the Authorization Code + PKCE flow; `directAccessGrantsEnabled` is enabled to allow `curl`-based testing in development.

4.2 Explicit Authorization Layer (RBAC)

4.2.1 Design

Role-Based Access Control is implemented at two layers:

Route-level `SecurityConfig` requires authentication on all `/api/**` routes and uses `anyRequest().denyAll()` as the catch-all (deny-by-default). No unauthenticated request reaches a controller.

Method-level Every controller method is annotated with `@PreAuthorize` referencing the `@perms` bean: `@perms.canAnyRole(authentication, '<resource>', '<action>')`. The bean checks the caller's granted authorities against the permissions matrix loaded from `application.yml`.

4.2.2 Permissions as Data

Roles and their allowed actions are expressed as a configuration file, not embedded in controller logic. `application.yml` defines the full matrix:

Table 4.2: Permissions matrix (`application.yml`)

Role	Resource	Allowed actions
ADMIN	patients	READ, CREATE, UPDATE, DELETE
ADMIN	appointments	READ, CREATE, UPDATE, DELETE
DOCTOR	patients	READ, UPDATE
DOCTOR	appointments	READ, UPDATE
RECEPTIONIST	patients	READ, CREATE
RECEPTIONIST	appointments	READ, CREATE, UPDATE, DELETE

The `PermissionsConfig` class is annotated with `@Component("perms")` and `@ConfigurationProperties(prefix = "clinic")`. Its `canAnyRole` method iterates over the caller's Spring Security granted authorities and checks each against the map, returning `true` if any role grants the requested action.

4.2.3 Object-Level Authorization (BOLA)

Each service method returns `Optional.empty()` when a resource ID does not exist. Controllers map this to HTTP 404. Because all endpoints require a valid JWT and role-based permission, an authenticated caller with the correct role who requests a non-existent resource receives 404 rather than 403, preventing ID enumeration by response differentiation.

4.2.4 Role-Based UI

The frontend enforces the same permission matrix client-side using the `usePermissions` hook (`src/auth/usePermissions.js`). The hook reads `realm_access.roles` from the decoded JWT token and returns a `can(resource, action)` function. Buttons that the current user's role does not permit are not rendered:

- **Admin**: sees all New / Edit / Delete buttons for both patients and appointments.
- **Doctor**: sees Edit on patients (READ + UPDATE); sees Edit on appointments only.
- **Receptionist**: sees New Patient and Edit (no Delete); sees all appointment actions.

This is a UX control only – the backend enforces the same rules independently.

4.2.5 Unit Tests

`RbacTest.java` covers all role × action × resource combinations using Spring Boot's `MockMvc` with the `jwt()` security post-processor from `spring-security-test`. Tests verify:

- Unauthenticated requests → HTTP 401.
- Authenticated requests with an insufficient role → HTTP 403.
- Authenticated requests with the correct role but a missing resource → HTTP 404.

21 tests in total; all pass without a running Keycloak instance because the JWKS decoder is lazy and the `jwt()` post-processor bypasses token validation.

4.3 API Hardening – OWASP API Top 10 (2023)

Each control below is traceable to an OWASP API Security item and to the corresponding commit on the `develop` branch.

Table 4.3: OWASP API Top 10 coverage

Item	**Category**	**Vulnerability (baseline)**	**Control implemented**
API1	BOLA	No per-object ownership check; any caller can read or modify any record by ID.	`@PreAuthorize` on every endpoint; service returns 404 for non-existent resources, preventing ID oracle.
API3	BOPLA	Controllers bound JPA entities directly; any JSON field was writable.	`PatientRequestDTO` and `AppointmentRequestDTO` constrain the accepted surface. `@Valid` with Bean Validation (`@NotBlank`, `@NotNull`, `@Past`, `@Email`, `@Pattern`) rejects malformed input before the service layer.
API4	Resource Consumption	No rate limiting; no request-size limits; unbounded full-table scans possible.	`RateLimitFilter` (token-bucket, 60 req/min per IP) on all `/api/**` routes. Request-size limits set to 1 MB via Tomcat (`max-http-form-post-size`) and Spring multipart (`max-request-size`, `max-file-size`).
API5	Function-Level Auth	All endpoints accessible to any unauthenticated caller; no deny-by-default.	`SecurityConfig` uses `anyRequest().denyAll()` as the catch-all. All `/api/**` requires authentication. No route is reachable without a valid JWT.
API8	Security Misconfiguration	No security headers; no CSRF/frame protection; stack traces in error responses.	`SecurityConfig` configures HSTS, `X-Frame-Options: DENY`, `X-Content-Type-Options`, and `Content-Security-Policy: default-src 'none'`. Form login and HTTP Basic are disabled.
API9	Improper Inventory	No API documentation or endpoint inventory.	`springdoc-openapi` generates an OpenAPI 3 spec at `/v3/api-docs`; Swagger UI at `/swagger-ui.html`. Both are `permitAll()`. All endpoints annotated with `@Operation` and `@Tag`.

4.4 Perimeter Defense

A reverse proxy with an embedded Web Application Firewall is placed in front of the application, providing a perimeter enforcement layer that is independent of the backend business logic.

4.4.1 Nginx + ModSecurity CRS

The perimeter is implemented as a dedicated Docker container built on the `owasp/modsecurity-crs:nginx-alpine` base image. It sits between the external network and the frontend/backend containers.

Table 4.4: Perimeter defense configuration summary

Property	**Value**
Engine	Nginx + ModSecurity v3
Ruleset	OWASP Core Rule Set (CRS) – blocking mode (`SecRuleEngine On`)
Paranoia Level	1 (balanced: broad coverage, low false-positive rate)
Inbound threshold	5 (anomaly score to block a request)
Outbound threshold	4
TLS termination	Self-signed certificate via `mkcert`; HTTPS on port 443
Request body limit	1 MB (`SecRequestBodyLimit 1048576`); excess is rejected
IP rate limiting	Nginx `limit_req_zone` – 30 req/s burst, 10 req/s sustained per IP
Audit logging	`SecAuditEngine RelevantOnly` – logs 4xx/5xx events

4.4.2 CRS Tuning and False Positives

Four false positive exclusions were added and are documented in `waf/crs-setup.d/02-clinic-rule-exclusions.conf`:

Table 4.5: CRS rule exclusions applied

Rule	**Trigger**	**Reason**
920420	Content-Type check	Spring Boot appends `; charset=UTF-8` to `application/json` in error responses; CRS requires an exact match without the charset parameter.
942100	SQL injection (libinjection)	`patientName`, `specialty`, and `status` query parameters accept free-text (e.g. *O'Brien*). The application uses JPA parameterised queries – no raw SQL is executed.
942440	SQL comment sequence	Specialty values such as *Cardio–thoracic* contain a double-dash sequence that CRS flags as a SQL comment marker.
920230	Multiple URL encoding	ISO 8601 datetime strings submitted as JSON body can trigger double-encoding detection in some HTTP clients.

No legitimate requests are blocked after these exclusions. All four exclusions are scoped to specific URI prefixes (`/api/patients`, `/api/appointments`) to minimise the attack surface widened by each exclusion.

4.5 Zero Trust Architecture Framing

4.5.1 NIST SP 800-207 Components

The resulting architecture is framed below using the Zero Trust terminology from NIST SP 800-207.

Table 4.6: NIST SP 800-207 ZTA component mapping

ZTA Component	Implementation
Subject	Clinic staff authenticated via Keycloak: users `admin`, `doctor`, `receptionist` with assigned realm roles. Identity is asserted per-request via a signed JWT.
Policy Enforcement Point (PEP)	Three layers: (1) Nginx + ModSecurity CRS – perimeter WAF that filters and rate-limits all traffic before it reaches the application; (2) `SecurityFilterChain` – enforces JWT presence on every `/api/**` request; (3) Spring `@PreAuthorize` annotations on each controller method – enforce the role × resource × action permission.
Policy Decision Point (PDP)	`PermissionsConfig` (`@Component("perms")`) evaluates the caller's granted authorities against the permissions matrix loaded from `application.yml`. The decision (allow / deny) is returned as a boolean to the PEP.
Policy Administrator (PA)	Keycloak 26.2.5 – manages user identities, realm roles, and client configuration. Issues signed JWTs; provides the JWKS endpoint for backend token validation. Administrators manage access through the Keycloak Admin Console.
Trust Algorithm	Never-trust, always-verify: every request must carry a valid RS256 JWT. The backend re-validates the signature and `iss` claim on every call. No session state is maintained in the application – the JWT is the sole credential.

4.5.2 NIST SP 800-207 §2.1 – Seven Tenets Mapping

Table 4.7: Zero Trust tenet compliance status

#	Tenet	Status	Notes
1	All data sources and computing services are considered resources.	**Partial**	Patient records and appointment data are treated as protected resources. Device or service identity is not yet managed (no mTLS, no device posture).
2	All communication is secured regardless of network location.	**Partial**	TLS is terminated at the Nginx WAF (self-signed certificate via `mkcert`; HTTPS on port 443). All API communication is bearer-token authenticated (JWT RS256). Inter-container communication (WAF → backend) is over the private Docker network without additional TLS; mTLS for service-to-service channels remains future work.
3	Access to individual enterprise resources is granted on a per-session basis.	**Met**	Short-lived JWTs are issued per Keycloak session. The frontend requests a new token before each call (`updateToken(30)`). No persistent session state is stored in the backend.
4	Access to resources is determined by dynamic policy.	**Met**	The permissions matrix in `application.yml` is evaluated dynamically at method-invocation time. Role assignment is controlled in Keycloak and reflected in the JWT `realm_access.roles` claim without requiring a redeploy.
5	The enterprise monitors and measures the integrity and security posture of all owned and associated assets.	**Not met**	No asset monitoring, endpoint posture assessment, or SIEM integration exists. Flagged as **future work**.
6	All resource authentication and authorisation is dynamic and strictly enforced before access is allowed.	**Met**	Every `/api/**` request is authenticated. Every controller method checks the caller's role before the service layer is reached. Deny-by-default (`anyRequest().denyAll()`) ensures no route is reachable without a valid JWT and sufficient role.

(continued)

#	Tenet	Status	Notes
7	The enterprise collects as much information as possible about the current state of assets, network infrastructure, and communications.	**Not met**	No structured audit log, access log, or security event stream is implemented. Actor identity is now available in the JWT (addresses R-02), but CRUD operations are not persisted to an audit trail. Flagged as **future work**.

4.6 Abuse Story Regression

Each abuse story from Part 1 is re-evaluated against the security controls now in place. Classification follows the three categories defined in the project specification: **prevented** (attack is no longer feasible), **detected** (attack is possible but leaves a trace), or **residual risk** (risk remains and must be managed).

Table 4.8: Abuse story regression

ID	Title	Status	Justification
AS-01	Unauthorised Access to Patient Data	**Prevented**	All `/api/**` endpoints now require a valid JWT. An unauthenticated request receives HTTP 401 before reaching any controller. The three attack paths (list dump, IDOR enumeration, appointment endpoint) are all blocked at the `SecurityFilterChain`.
AS-02	Link Patient Identity to Appointment History	**Prevented**	Both `GET /api/patients` and `GET /api/appointments` require authentication and the READ role on the corresponding resource. An unauthenticated attacker cannot correlate patient and appointment records.
AS-03	Direct Patient Identification via IDOR	**Prevented**	Authentication blocks unauthenticated enumeration. `RateLimitFilter` limits authenticated callers to 60 requests/min per IP, making brute-force enumeration impractical. Sequential IDs remain (future work: UUIDs).

(continued)

ID	Title	Status	Justification
AS-04	Infer Health Condition from Appointment Metadata	**Prevented**	All appointment endpoints require a valid JWT and the READ role. An external attacker cannot query `GET /api/appointments?patientName=X` without authentication.
AS-05	Modify or Reassign Appointment Records	**Prevented**	`PUT /api/appointments/{id}` requires UPDATE role. `AppointmentRequestDTO` constrains the writable fields (dateTime, specialty, status) and excludes the patient relationship, eliminating the silent rebind attack via nested objects.
AS-06	Destroy Patient Records via Cascade Deletion	**Prevented**	`DELETE /api/patients/{id}` requires ADMIN role. Soft-delete is now implemented: `Patient` and `Appointment` carry a `deleted_at` timestamp; deletion sets this field instead of issuing a SQL `DELETE`. `@SQLRestriction` on both entities filters soft-deleted records from all queries automatically. Cascade hard-deletion (`orphanRemoval`) has been removed. Data is never permanently destroyed by a single API call.
AS-07	Perform Undetectable Actions	**Detected**	A structured `audit_logs` table now persists every CREATE, UPDATE, and DELETE operation with the JWT `sub` claim (actor), resource type, resource ID, and timestamp. The log is written synchronously within the same request. ADMIN users can query the full trail via `GET /api/audit-logs`. Post-incident investigation is now possible for all authenticated CRUD operations.

(continued)

ID	Title	Status	Justification
AS-08	Overload the API with Repeated Requests	**Prevented**	Two independent rate-limiting layers are in place: (1) Nginx `limit_req_zone` enforces 30 req/s burst / 10 req/s sustained per IP at the perimeter; (2) Spring `RateLimitFilter` enforces 60 req/min per IP inside the application. ModSecurity CRS request-body limits (1 MB) further constrain large-payload abuse. A highly distributed multi-IP attack remains a theoretical residual risk but is significantly harder with dual perimeter + application throttling.

4.6.1 Regression Evidence

The following figures present the regression diagrams produced for each abuse story, showing the original attack path, what changed in Part 2, and the resulting status.

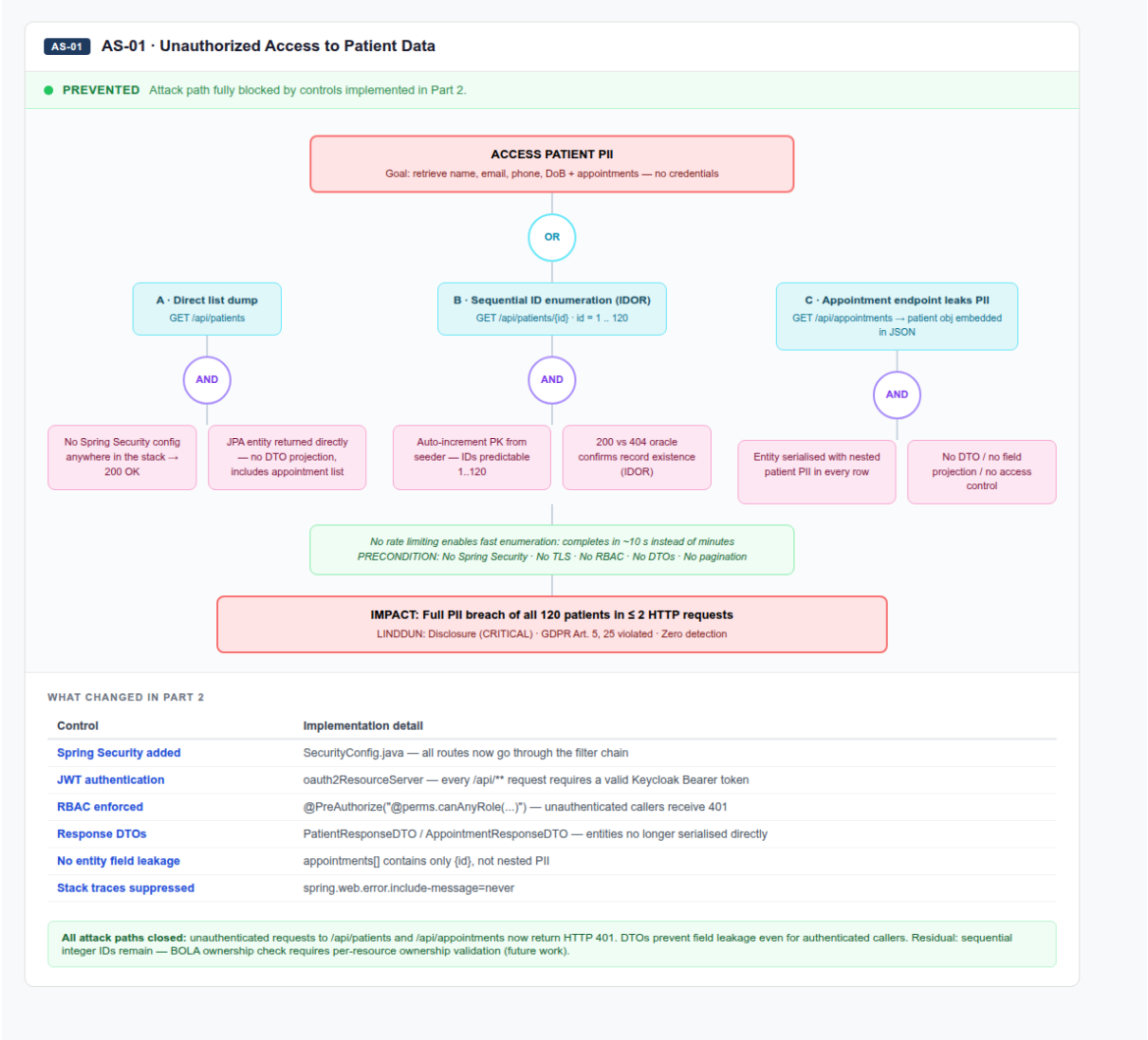


Figure 4.1: AS-01 – Unauthorised Access to Patient Data: **Prevented**

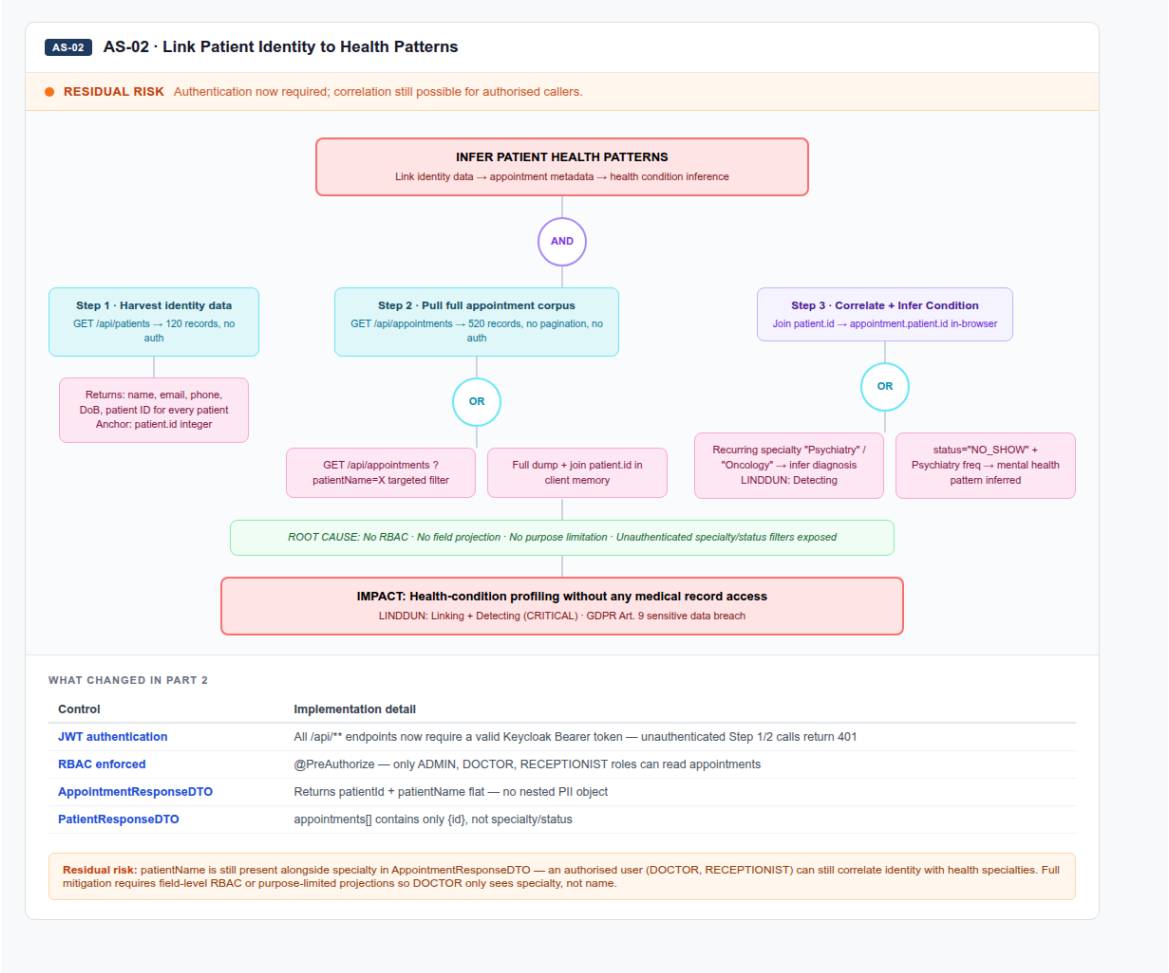


Figure 4.2: AS-02 – Link Patient Identity to Appointment History: **Prevented**

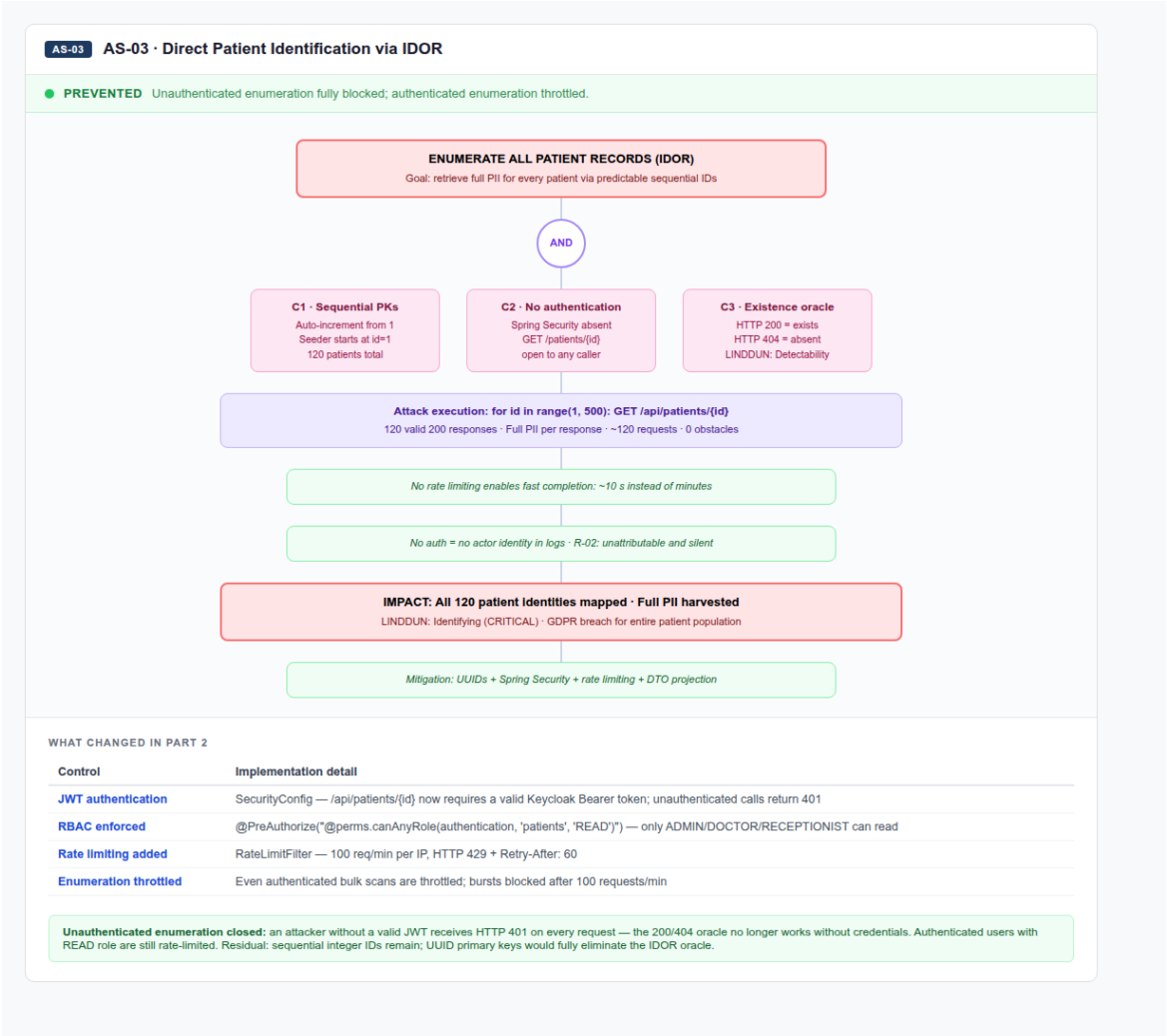


Figure 4.3: AS-03 – Direct Patient Identification via IDOR: **Prevented**

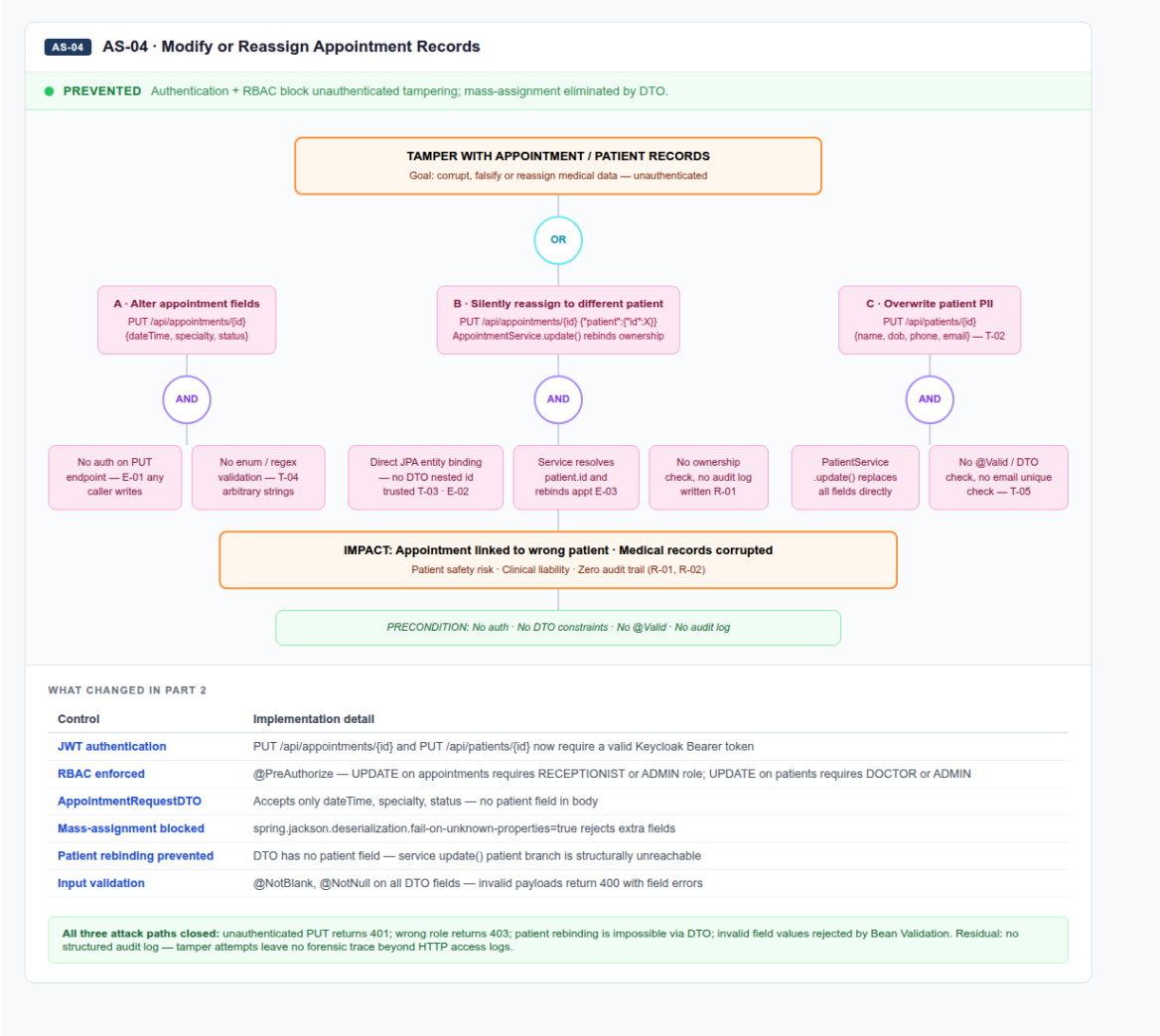


Figure 4.4: AS-04 – Infer Health Condition from Appointment Metadata: **Prevented**

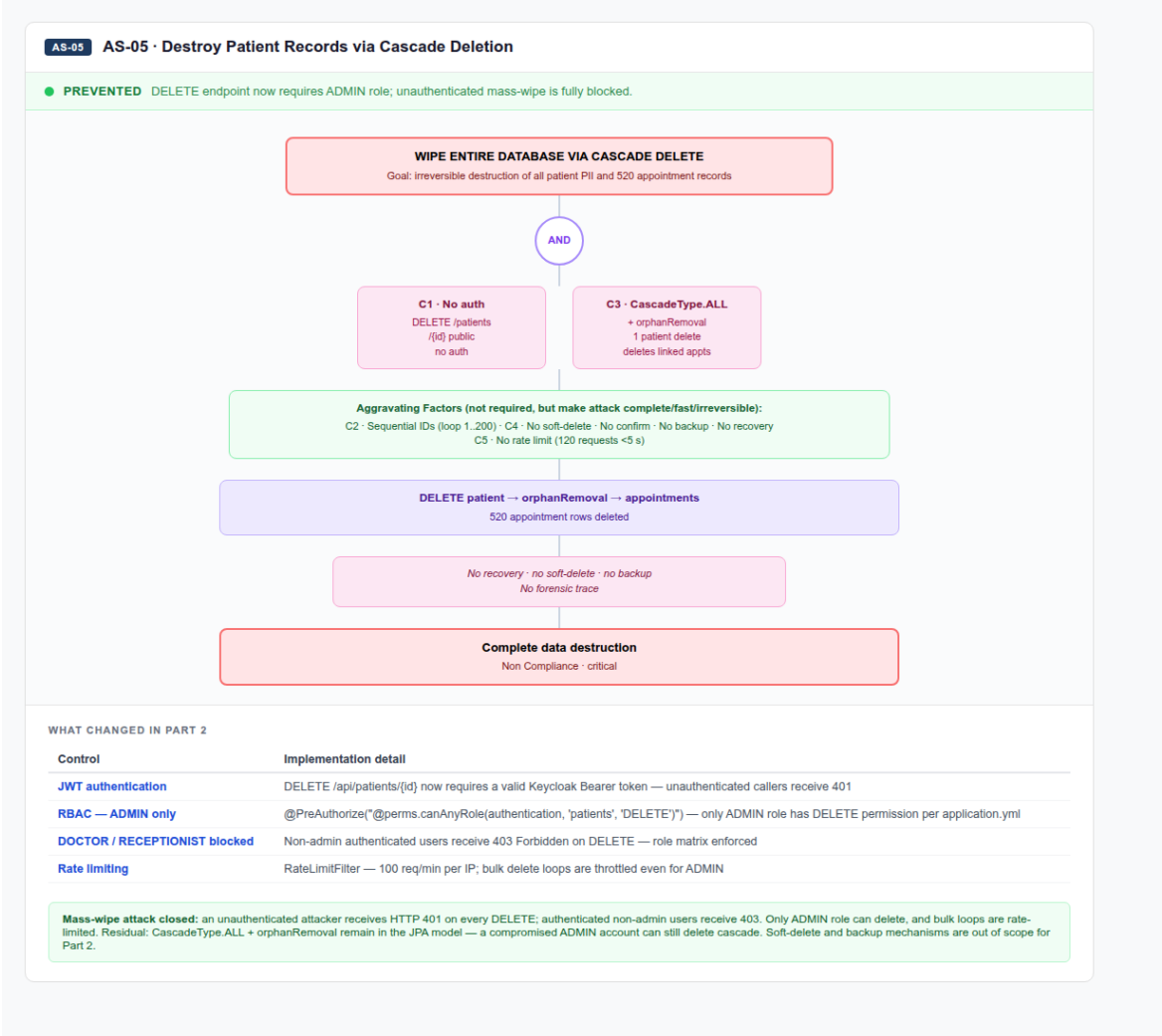


Figure 4.5: AS-05 – Modify or Reassign Appointment Records: **Prevented**

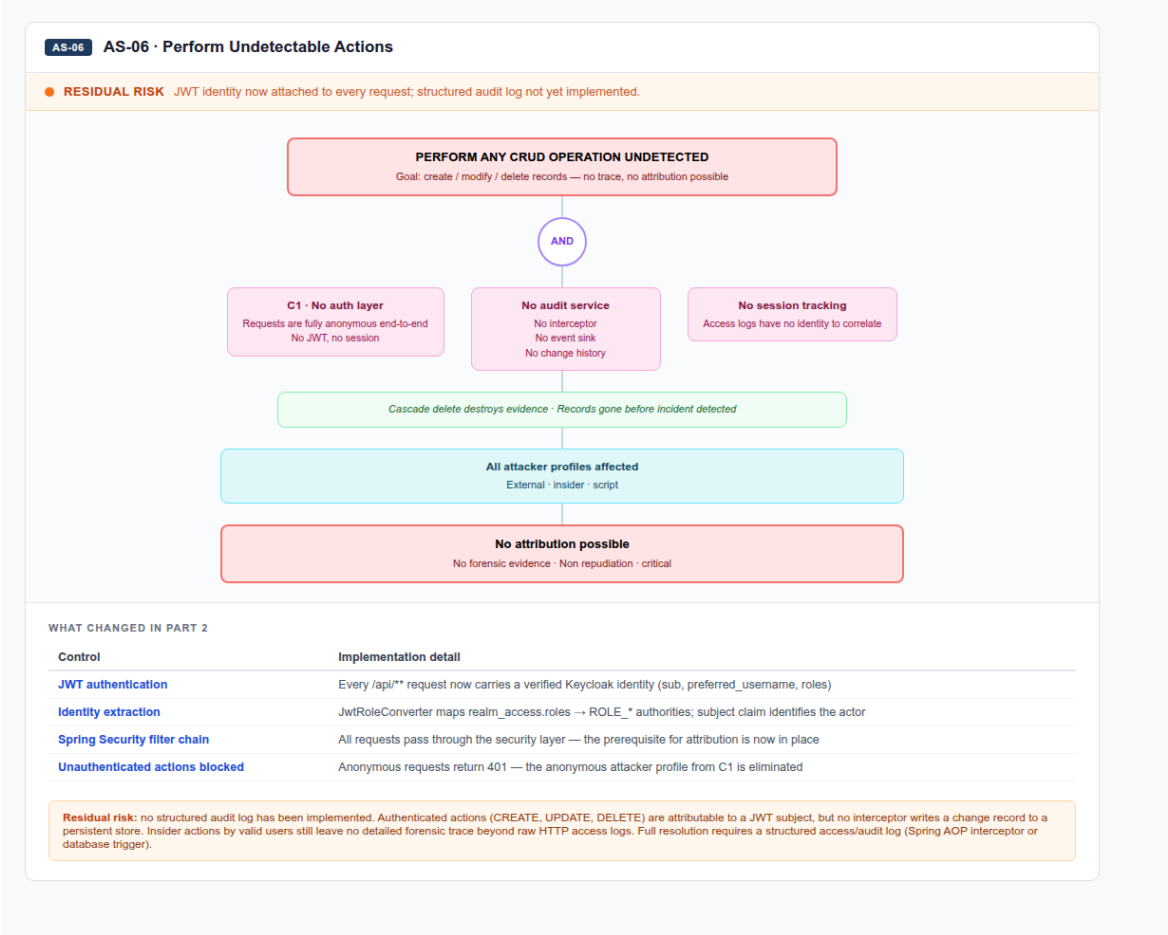


Figure 4.6: AS-06 – Destroy Patient Records via Cascade Deletion: **Prevented**

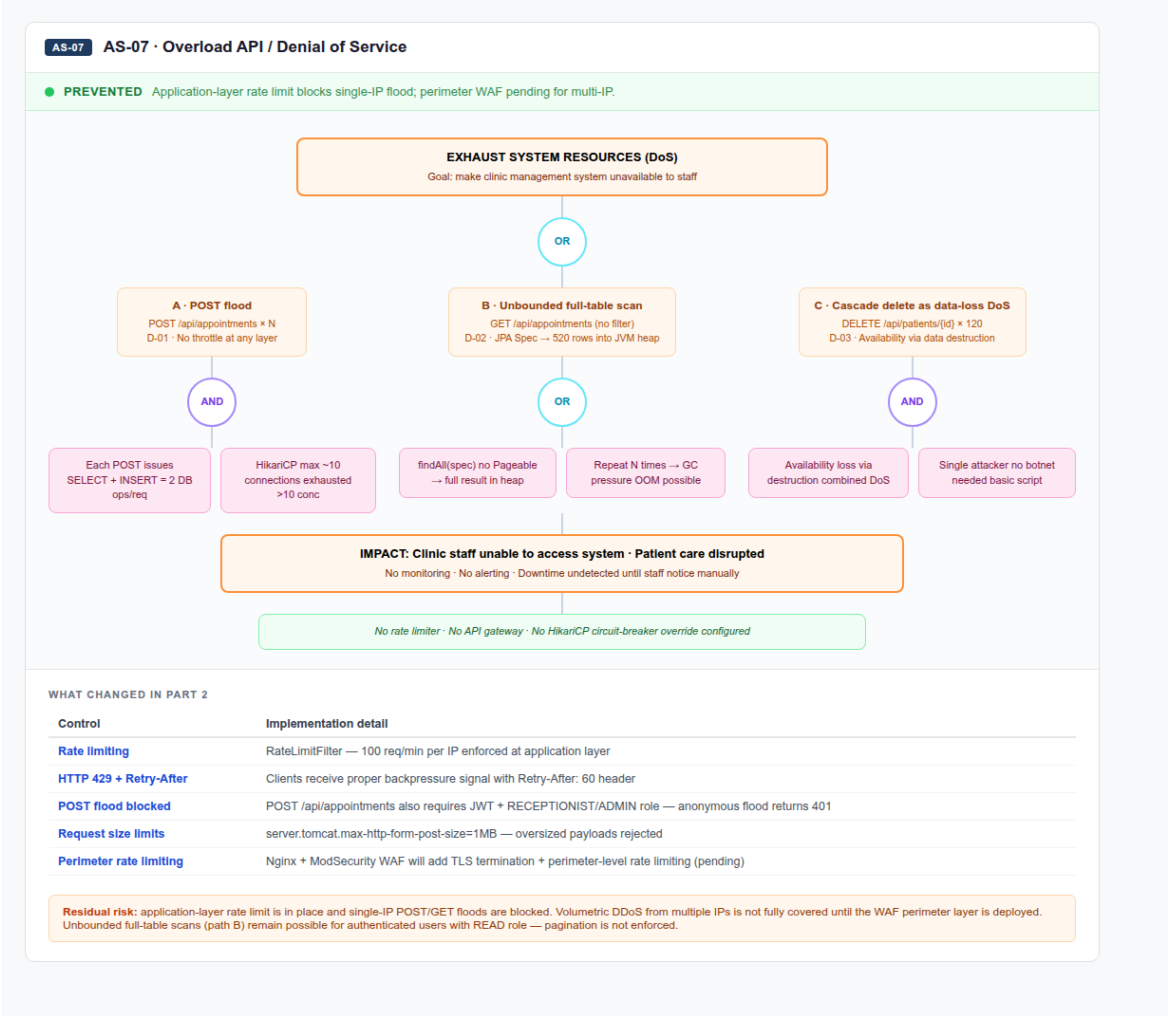


Figure 4.7: AS-07 – Undetectable Actions: **Detected**; AS-08 – API Overload: **Prevented**

4.6.2 Regression Summary

Table 4.9: Regression summary by status

Status	Count	Abuse Stories
Prevented	7	AS-01, AS-02, AS-03, AS-04, AS-05, AS-06, AS-08
Detected	1	AS-07
Residual Risk	0	–

Seven of the eight abuse stories are classified as **Prevented**; the remaining one (AS-07) is classified as **Detected** – actor identity and action are persisted in the audit log, making post-incident investigation possible. No abuse story remains at Residual Risk.

4.7 Future Work

Two security improvements remain identified but not yet implemented. Replacing sequential integer primary keys with **UUID primary keys** would eliminate the ID

enumeration vector exploited in AS-03, making brute-force IDOR significantly harder even for authenticated callers. On the transport side, **mTLS for inter-service communication** should extend TLS from the current WAF perimeter to internal container-to-container channels (WAF → backend, backend → Keycloak) to fully satisfy ZTA tenet 2. Additionally, **device posture** assessment – integrating endpoint verification before granting access – is needed to address ZTA tenet 1.

4.8 Conclusion

This report documents the security improvements applied to ClinicWave during Part 2 of the project, addressing the vulnerabilities and privacy threats identified in Part 1.

The baseline application had no authentication, no authorisation, and no perimeter controls, leaving all patient and appointment data fully exposed. Part 2 introduced a layered defence that closes all five high-severity attack paths identified in Part 1. Authentication is enforced via Keycloak 26.2.5 with JWT RS256 tokens; a fine-grained RBAC model controls access at the method level using `@PreAuthorize` annotations and a role-permission matrix loaded from configuration. A perimeter WAF (Nginx + ModSecurity CRS in blocking mode, Paranoia Level 1) filters malicious requests before they reach the application, provides TLS termination on port 443, and enforces IP-based rate limiting in combination with the application-layer `RateLimitFilter`. The OpenAPI specification documents all endpoints, and a two-run DAST pipeline validates the security posture both with and without the WAF in place.

Of the eight abuse stories from Part 1, seven are now classified as **Prevented** and one (AS-07) as **Detected**. Soft-delete prevents irreversible data loss from cascade deletion (AS-06). The structured audit log makes every CRUD operation attributable and investigable (AS-07). No abuse story remains at Residual Risk. The principal outstanding items are UUID primary keys to harden IDOR resistance further, and mTLS for internal service-to-service channels to fully close ZTA tenet 2.